

N036 뉴스레터 에이전트 만들기_해설서

Chapter01

이번 챕터는 "뉴스를 자동으로 찾아주는 AI"를 만드는 것보다, 더 중요한 "복잡한 업무를 여러 단계로 쪼개고 각 단계를 하나의 노드로 설계하는 방법"을 배우는 챕터입니다.

/강의정리 챕터 01

뉴스레터 에이전트 — 수집부터 발행까지

1. 이번 강의에서 만들 것은 무엇인가?

매일 AI 분야에는 수많은 뉴스가 쏟아집니다.

사람이 직접 한다면:

| 뉴스 검색 → 읽기 → 중요한 것 고르기 → 요약 → 사실 확인 → 뉴스레터 작성 → 발행

이라는 과정을 매일 반복해야 합니다.

이번 강의에서는 이 과정을 **AI 뉴스레터 에이전트**로 자동화합니다.

핵심은 단순히 AI에게

| "오늘 AI 뉴스 알려줘."

라고 하는 것이 아닙니다.

전체 업무를 **5개의 단계로 분해**하고, 각각을 하나의 **노드(Node)**로 만드는 것입니다.

전체 구조

- ① 자료 수집
- ↓
- ② 중요도 선별
- ↓
- ③ 요약
- ↓
- ④ 검수

↓
⑤ 발행

이것이 이번 강의의 가장 중요한 구조입니다.

2. 왜 일을 쪼개야 할까?

'뉴스레터 에이전트'라고 생각하면 굉장히 복잡해 보입니다.

하지만 일을 잘게 나누면 단순합니다.

예를 들어 식당에서 음식을 만드는 것과 비슷합니다.

재료 준비
↓
좋은 재료 선별
↓
요리
↓
맛 검사
↓
손님에게 제공

뉴스레터도 똑같습니다.

뉴스 가져오기
↓
좋은 뉴스 고르기
↓
요약하기
↓
검사하기
↓
독자에게 보내기

즉, 복잡한 하나의 업무를 작은 업무들의 연결로 바꾸는 것입니다.

이것이 AI Agent와 Workflow를 설계할 때 매우 중요한 사고방식입니다.

3. 뉴스레터의 5단계

① 자료 수집

가장 먼저 질문해야 합니다.

어디에서 뉴스를 가져올 것인가?

뉴스를 가져오는 방법은 여러 가지가 있습니다.

예를 들어:

- RSS
- 공개 API
- 웹 검색
- 특정 웹사이트
- 블로그
- Hacker News 등

각각 장단점이 있습니다.

따라서 단순히 "뉴스를 많이 가져오자"가 아니라

신뢰할 수 있고 안정적으로 가져올 수 있는가?

를 판단해야 합니다.

핵심

좋은 AI 결과는 좋은 입력에서 시작됩니다.

4. ② 중요도 선별

뉴스를 100개 가져왔다고 해서 100개를 모두 독자에게 보여주면 좋은 뉴스레터가 아닙니다.

오히려 독자는 피곤해집니다.

따라서

100개 뉴스
↓
중요도 평가
↓
20개
↓
다시 평가

↓
3~5개

처럼 줄여야 합니다.

이 과정에서는 **예선** → **본선** 구조를 사용할 수 있습니다.

예선

많은 뉴스 중에서 가능성이 있는 뉴스를 빠르게 걸러냅니다.

본선

남은 뉴스들을 좀 더 엄격한 기준으로 비교하여 최종 3~5개를 선택합니다.

즉,

↓
AI에게 무조건 하나씩 판단시키는 것이 아니라, 선별 과정을 설계하는 것
이 중요합니다.

5. ③ 요약

선별된 뉴스는 그대로 보여주는 것이 아니라 독자가 빠르게 이해할 수 있도록 요약합니다.

여기서 중요한 것은 단순한 "짧게 줄이기"가 아닙니다.

예를 들어 AI 뉴스라면 다음처럼 요약할 수 있습니다.

무슨 일이 일어났나?
↓
왜 중요한가?
↓
무엇이 달라지는가?
↓
우리에게 어떤 의미가 있는가?

즉, **어떤 관점으로 요약할 것인지를** 먼저 정해야 합니다.

이것이 프롬프트 설계와 출력 스키마 설계로 연결됩니다.

6. ④ 검수

AI가 요약했다고 해서 바로 발행하면 위험합니다.

AI는 다음과 같은 실수를 할 수 있습니다.

- 원문에 없는 내용을 추가
- 숫자를 잘못 전달
- 회사명이나 제품명 오류
- 서로 다른 기사를 섞음
- 원문의 의미를 과도하게 해석

따라서

```
원문
↓
AI 요약
↓
검수
↓
통과 → 발행
실패 → 제외/수정
```

라는 구조가 필요합니다.

여기서 중요한 개념이 **검수 기준**입니다.

"좋은 요약인지 확인해줘"처럼 막연하게 검사하는 것이 아니라,

무엇과 대조해서 무엇을 검사할 것인가?

를 명확하게 정해야 합니다.

7. ⑤ 발행

마지막은 결과를 실제 사용자에게 전달하는 단계입니다.

이번 강의에서는 **Discord**를 이용합니다.

그리고 **GitHub Actions**를 이용하여 자동 실행합니다.

최종적으로는 이런 구조가 됩니다.

```
매일 아침
↓
GitHub Actions 실행
↓
뉴스 수집
↓
```

중요 뉴스 선별
↓
요약
↓
검수
↓
Discord 발행

그러면 사람이 매일 직접 실행하지 않아도 됩니다.

자동화가 실제 생활 속에서 작동하는 단계까지 만드는 것입니다.

8. LangGraph에서는 어떻게 표현할까?

앞선 강의에서 배운 **LangGraph**를 이용하면 각각의 업무를 Node로 만들 수 있습니다.

개념적으로는 다음과 같습니다.

```
[수집 Node]
  ↓
[선별 Node]
  ↓
[요약 Node]
  ↓
[검수 Node]
  ↓
[발행 Node]
```

각 Node가 하나의 역할을 담당합니다.

Node란?

쉽게 말하면 **일을 하는 작업자**입니다.

예를 들어:

- 수집 Node → 뉴스를 가져오는 직원
- 선별 Node → 중요한 뉴스를 고르는 직원
- 요약 Node → 기사를 정리하는 직원
- 검수 Node → 결과를 검사하는 직원
- 발행 Node → 독자에게 보내는 직원

이렇게 생각하면 LangGraph의 구조가 훨씬 쉽게 이해됩니다.

9. 이번 강의의 전체 설계

강의는 크게 다음과 같이 진행됩니다.

단계	섹션	핵심 내용
준비	1~2	5단계를 정하고 빈 노드로 뼈대 구성
자료 수집	3~5	뉴스 경로 선정 및 수집 노드 제작
중요도 선별	6~7	예선·본선 구조로 뉴스 선별
요약	8~9	취재 노드와 출력 형식 설계
검수	10	원문과 비교하여 검수
발행	11~12	Discord + GitHub Actions 자동화
운영	13~15	지표 측정 및 다른 분야로 확장

여기서 중요한 것은 **처음부터 완성된 AI Agent를 만들려고 하지 않는다는 것**입니다.

문제 정의
→ 단계 분해
→ 노드 설계
→ 각 노드 구현
→ 연결
→ 전체 실행
→ 결과 측정
→ 개선

이라는 방식으로 만들어갑니다.

10. 100개의 뉴스가 5개의 뉴스가 되는 과정

이번 실습의 결과를 보면 약 **50~120개의 뉴스**가 들어올 수 있습니다.

그리고 최종적으로 3~5개 정도만 남깁니다.

예를 들어:

RSS / API 등
↓
약 100개 수집
↓
중복·관련성 등 1차 필터
↓
중요도 평가
↓
후보 뉴스

↓
최종 선별
↓
5개
↓
요약
↓
검수
↓
Discord

이것은 단순한 요약 AI가 아니라 정보 처리 파이프라인입니다.

11. 이 강의에서 특히 중요한 개념

① 수집보다 중요한 것은 '소스의 품질'

AI에게 아무 자료나 주면 좋은 결과가 나오기 어렵습니다.

따라서

| 무엇을 가져올 것인가?

가 첫 번째 설계 문제입니다.

② AI에게 판단을 한 번에 맡기지 않는다

100개를 한 번에 읽고 5개를 고르라고 하는 것보다

1차 선별
→ 2차 선별
→ 최종 선택

처럼 여러 단계로 나누는 것이 더 관리하기 쉽습니다.

③ 요약에는 '관점'이 필요하다

단순히 기사를 줄이는 것이 아니라

| "이 뉴스가 왜 중요한가?"

를 설명하도록 만들 수 있습니다.

이 부분이 나중에 **ASP의 AI 컨설팅**이나 **콘텐츠 제작**과도 직접 연결됩니다.

④ 검수는 선택이 아니라 필수

AI가 만들어낸 결과를 그대로 믿지 않고

원자료 → 결과물

을 비교하는 구조를 만드는 것이 중요합니다.

⑤ 자동화의 완성은 '실제 실행'

코드가 실행되는 것만으로 끝나는 것이 아닙니다.

실행 성공

에서 끝나는 것이 아니라

뉴스가 제대로 수집됨
→ 좋은 뉴스가 선택됨
→ 정확하게 요약됨
→ 검수를 통과함
→ 실제로 발행됨

까지 가야 업무 자동화가 성공한 것입니다.

12. 이 강의가 AI 개발자에게 중요한 이유

이번 강의는 단순히 "뉴스레터 만드는 법"을 배우는 과정이 아닙니다.

AI 개발에서 매우 중요한 **Agent Workflow 설계 방법**을 연습합니다.

앞으로 어떤 업무를 자동화하든 다음 질문을 할 수 있어야 합니다.

- ① 이 업무의 시작은 무엇인가?
- ② 어떤 단계로 나눌 수 있는가?
- ③ 각 단계의 입력과 출력은 무엇인가?
- ④ 어느 단계에 AI가 필요한가?
- ⑤ 어디에서 검수가 필요한가?
- ⑥ 최종적으로 무엇이 나오면 성공인가?

이 질문을 할 수 있으면 뉴스레터뿐 아니라 **컨설팅, 콘텐츠 제작, 리서치, 고객 분석, 보고서 작성**에도 적용할 수 있습니다.

13. 사용자님의 1인 창업 관점에서 보면

이 챕터는 특히 **ASP(AI Story Partner)**를 만들 때 중요한 기초가 됩니다.

예를 들어 나중에 고객에게

|"우리 회사 업무를 AI로 자동화하고 싶습니다."

라는 의뢰를 받았다고 생각해보겠습니다.

바로 개발부터 시작하는 것이 아니라,

```
현재 업무
↓
업무 분해
↓
반복 업무 발견
↓
AI가 할 일
↓
사람이 검수할 일
↓
자동화 Workflow
↓
성과 측정
```

으로 분석할 수 있어야 합니다.

즉, 이번 뉴스레터 실습은 **작은 자동화 프로젝트를 통해 AI 업무 설계 능력을 연습하는 과정**이라고 이해하시면 좋습니다.

특히 사용자님이 과거에 해오셨던 **신사업 기획·프로젝트 PM·서비스 기획 경험**과도 상당히 잘 맞는 영역입니다. 개발자가 아니더라도 **"어떤 일을 어떤 구조로 자동화할 것인가"**를 설계하는 능력이 핵심이기 때문입니다.

★ 이번 챕터 핵심 정리

뉴스레터 에이전트의 5단계

수집 → 선별 → 요약 → 검수 → 발행

전체 사고방식

하나의 복잡한 업무를 여러 개의 작은 Node로 분해한다.

중요한 설계 원칙

이것만은 꼭 기억하세요

1. Agent를 만들기 전에 먼저 업무를 쪼갬다.
 2. 쪼갬 업무 하나하나가 Workflow의 Node가 된다.
 3. AI가 판단한 결과에는 검수 단계가 필요하다.
 4. 실행되는 것과 실제 업무가 성공하는 것은 다르다.
 5. 자동화의 최종 목적은 'AI가 일했다'가 아니라 '사람의 일이 실제로 줄었다'이다.
-

한 문장으로 정리

뉴스레터 에이전트는 '뉴스를 찾아주는 AI'가 아니라, 수집 → 선별 → 요약 → 검수 → 발행이라는 하나의 업무를 여러 Node로 분해하여 자동으로 실행하는 AI Workflow입니다.

Chapter02

이번 챕터는 "AI 에이전트를 어떻게 만들 것인가"보다 "복잡한 일을 어떤 순서와 구조로 만들어야 실패를 줄일 수 있는가"를 배우는 내용입니다. 특히 LangGraph에서 State와 Node를 어떻게 설계하는지가 핵심입니다.

/강의정리 챕터 02

만드는 순서 정하기

1. 이번 챕터의 핵심 질문

챕터 01에서는 뉴스레터 업무를 5단계로 나눴습니다.

| 수집 → 선별 → 취재(요약) → 검수 → 발행

그렇다면 이제 이 5개를 어떤 순서로 개발할 것인가?

일반적인 방법은 다음과 같습니다.

수집 함수 완성
↓
선별 함수 완성
↓
요약 함수 완성
↓
검수 함수 완성
↓
발행 함수 완성
↓
마지막에 연결

문제는 **마지막에 연결하기 전까지 전체 시스템이 한 번도 제대로 돌아가지 않는다는 것**입니다.

그리고 마지막 조립 단계에서

┆ "어? 수집 결과의 이름과 선별 함수가 요구하는 이름이 다르네?"

같은 문제가 한꺼번에 발생합니다.

2. 이번 강의에서 선택한 방법

이번에는 반대로 합니다.

② 빈 노드로 뼈대를 먼저 만들고 하나씩 채운다

처음부터 완성된 함수를 만들지 않습니다.

먼저:

[수집]
↓
[선별]
↓
[취재]
↓
[검수]
↓
[발행]

이라는 **전체 파이프라인을 먼저 연결**합니다.

처음에는 아무 일도 하지 않는 빈 노드입니다.

그 다음:

수집 노드 완성



전체 실행

선별 노드 완성



전체 실행

취재 노드 완성



전체 실행

...

처럼 하나씩 진짜 기능으로 바꿉니다.

3. 왜 이 방법이 좋은가?

가장 큰 장점은 처음부터 끝까지 계속 실행해 볼 수 있다는 것입니다.

처음에는:

수집 0건
선별 0건
취재 0건
검수 0건
발행 0건

이지만 그래프 자체는 이미 완성되어 있습니다.

이후 수집을 만들면:

수집 100건
선별 0건
취재 0건
검수 0건
발행 0건

선별을 만들면:

수집 100건
선별 5건
취재 0건

검수 0건
발행 0건

이런 식으로 발전합니다.

즉,

■ 매번 전체 시스템을 실행하면서 조금씩 완성한다.

이것이 이번 챕터의 핵심 개발 방식입니다.

4. 네 가지 개발 방법 비교

방법	장점	단점
① 부품 완성 후 조립	각 함수를 독립적으로 만들기 쉬움	마지막에 연결 문제가 한꺼번에 발생
② 빈 노드 → 하나씩 완성	처음부터 전체 실행 가능, 연결 문제 조기 발견	처음에 State 설계 필요
③ 노드마다 그래프 재작성	필요한 것만 그때그때 설계	그래프 코드 반복·수정 증가
④ 노트북 셀 순차 실행	가장 단순하고 탐색하기 쉬움	나중에 자동화할 때 다시 만들어야 함

이번 실습에서는 ②번을 선택합니다.

5. 핵심 개념 ① State

이번 챕터에서 가장 중요한 개념 중 하나입니다.

LangGraph에서 **State**는 노드와 노드 사이를 이동하는 공용 작업 공간이라고 생각하면 쉽습니다.

예를 들어 뉴스 기사 하나가 있다고 해보겠습니다.

처음에는:

제목
링크
출처
시간
요약

정도의 정보가 필요합니다.

따라서 기사 정보를 일정한 형태로 통일합니다.

기본 기사 State

```
title
url
source
at
summary
```

그리고 이후 단계에서 필요한 정보가 추가됩니다.

```
headline
summary
why
```

6. 왜 State의 형태를 통일해야 할까?

뉴스 소스마다 데이터 이름이 다릅니다.

예를 들어 어떤 곳은:

```
link
```

이라고 하고,

다른 곳은:

```
url
```

이라고 할 수 있습니다.

또 어떤 곳은:

```
id
```

를 사용할 수도 있습니다.

이것을 그대로 다음 단계에 전달하면 코드가 복잡해집니다.

```
if source == "A":
    link 사용

if source == "B":
    url 사용

if source == "C":
    id 사용
```

이런 코드가 계속 늘어나게 됩니다.

그래서 수집 단계에서 하나의 표준 형태로 바꿔줍니다.

```
외부 데이터
↓
[표준화]
↓
title / url / source / at / summary
↓
다음 노드
```

이렇게 하면 이후 노드는 뉴스가 어디에서 왔는지 신경 쓸 필요가 없습니다.

핵심

외부 세계의 제각각인 데이터를 내부에서는 하나의 규격으로 통일한다.

이것은 실제 AI 시스템을 만들 때도 매우 중요한 설계 원칙입니다.

7. State에 모든 것을 넣으면 안 된다

State에는 노드 사이를 실제로 전달해야 하는 정보만 넣습니다.

예를 들어:

넣어야 하는 것

```
기사 목록
선별된 기사
요약 결과
검수 결과
로그
```

넣지 않는 것

프롬프트 문자열
HTTP 세션
특정 함수 내부에서만 필요한 임시 변수

왜냐하면 State가 너무 커지면 모든 노드가 불필요한 정보를 들고 다니게 되기 때문입니다.
쉽게 말하면:

| **State**는 여러 작업자가 함께 사용하는 작업 문서이지, 각 작업자의 개인 서랍이 아닙니다.

8. State는 '공용 작업판'이라고 생각하면 쉽습니다

예를 들어 회사에서 뉴스레터를 만든다고 생각해보겠습니다.
책상 위에 이런 문서가 있습니다.

뉴스 작업판
수집된 기사
선정된 기사
작성된 요약
검수 결과
실행 로그

수집 담당자가 기사 목록을 올려놓으면,
선별 담당자가 그중 일부를 선택합니다.
그 다음 취재 담당자가 내용을 작성합니다.
검수 담당자가 확인합니다.
마지막으로 발행 담당자가 가져갑니다.
이 **공용 작업판**이 **LangGraph의 State**입니다.

9. Reducer란 무엇인가?

이번 챕터에서 중요한 또 하나의 개념입니다.

쉽게 말하면 여러 노드가 같은 State에 값을 쓸 때 어떻게 합칠 것인지 정하는 규칙입니다.

이번 실습에서는 `drafted` 와 `log` 에 Reducer를 사용합니다.

drafted

여러 취재 워커가 동시에 작업할 수 있기 때문입니다.

예를 들어:

```
워커 A → 기사 1 요약  
워커 B → 기사 2 요약  
워커 C → 기사 3 요약
```

각각의 결과를 합쳐야 합니다.

```
기사 1  
+ 기사 2  
+ 기사 3  
= drafted
```

따라서 **합치기(merge)**가 필요합니다.

log

각 노드가 로그를 하나씩 남깁니다.

```
수집 완료  
선별 완료  
취재 완료  
검수 완료  
발행 완료
```

따라서 로그도 기존 값에 계속 추가해야 합니다.

즉:

```
log + 새로운 log
```

가 필요합니다.

10. 그런데 collected와 picked에는 왜 Reducer가 없을까?

이것이 객관식 문제의 정답입니다.

정답: ②

`drafted` 는 여러 취재 워커가 동시에 쓰고, `log` 는 여러 노드가 계속 추가하기 때문에 합쳐야 합니다. 반면 `collected` 와 `picked` 는 한 노드가 쓰고 다음 노드가 읽으므로 새로운 값으로 덮어쓰면 됩니다.

예를 들어:

```
collected = 뉴스 100개
```

다음 단계에서:

```
picked = 뉴스 5개
```

가 됩니다.

여기에 잘못해서 Reducer를 적용하면 기존 결과와 새로운 결과가 합쳐져서

```
100 + 5 = 105개
```

처럼 되어버릴 수 있습니다.

즉,

언제 합칠 것인가? 언제 교체할 것인가?

를 State 설계에서 명확하게 해야 합니다.

11. 빈 노드는 왜 만드는가?

주관식 문제의 핵심입니다.

처음 만든 빈 노드는 실제로는 아무 일도 하지 않습니다.

수집 → 빈 값
선별 → 빈 값
취재 → 빈 값
검수 → 빈 값
발행 → 빈 값

그렇다면

|"어차피 나중에 지울 코드인데 왜 만들지?"

라는 생각이 들 수 있습니다.

하지만 이 빈 노드는 버리는 코드가 아니라 전체 시스템의 뼈대입니다.

12. 빈 노드가 실제로 하는 일

① 전체 연결 구조를 먼저 검증합니다.

수집
↓
선별
↓
취재
↓
검수
↓
발행

이 연결이 제대로 되는지 확인할 수 있습니다.

② 노드 사이의 규격을 미리 확인합니다.

수집이 무엇을 내놓고 선별이 무엇을 받는지 확인할 수 있습니다.

즉,

| 입력과 출력의 계약을 미리 검증하는 역할

을 합니다.

③ 현재 개발 상태를 알 수 있습니다.

로그를 보면:

수집 100건
선별 5건
취재 5건
검수 5건
발행 5건

처럼 어디까지 완성되었는지 바로 알 수 있습니다.

④ 나중에 문제가 생기는 범위를 좁힐 수 있습니다.

예를 들어:

수집 100
선별 5
취재 0

이라면 취재 단계에 문제가 있을 가능성이 높습니다.

전체 프로그램을 뒤질 필요가 없습니다.

13. '걷는 뼈대'라는 개념

이번 챕터에서 기억해둘 만한 표현입니다.

걸어 다니는 뼈대(Walking Skeleton)란

아주 간단하지만 시스템의 처음부터 끝까지 실제로 실행되는 최소한의 전체 구조를 말합니다.

처음부터 완벽한 시스템을 만드는 것이 아닙니다.

완벽한 수집
완벽한 선별
완벽한 요약
...

을 각각 만든 다음 연결하는 것이 아니라,

수집 - 선별 - 취재 - 검수 - 발행

을 먼저 **아주 얇게 연결해서 전체가 움직이게 만든 뒤**, 하나씩 살을 붙입니다.

이 방식은 소프트웨어 개발에서 상당히 강력한 방법입니다.

14. 너무 많이 미리 설계하지도 않는다

여기에도 중요한 균형점이 있습니다.

State를 처음부터 완벽하게 만들려고 하면 안 됩니다.

처음에는 몰랐던 정보가 나중에 필요해질 수 있습니다.

예:

```
처음
title
url
source
```

였다가 나중에

```
title
url
source
at
summary
headline
why
```

가 될 수 있습니다.

이것은 문제가 아닙니다.

중요한 것은

처음부터 모든 것을 맞히는 것이 아니라, 현재 단계에서 노드 간에 무엇을 주고받는지만 명확하게 하는 것

입니다.

15. `build()` 와 노드 교체

이번 코드에는 재미있는 구조도 있습니다.

`build()` 가 노드를 이름으로 찾아서 연결합니다.

예를 들어:

```
collect
pick
draft
verify
publish
```

라는 이름을 사용합니다.

그러면 처음에는:

```
collect = 빈 노드
```

였다가 다음 섹션에서 같은 이름으로 진짜 함수를 만들면,

```
collect = 실제 수집 함수
```

로 바뀔 수 있습니다.

즉, 전체 그래프 구조를 다시 뜯어고치지 않고 노드의 구현만 교체할 수 있습니다.

이것이 이번 강의의

|"뼈대는 유지하고 내부 기능을 하나씩 갈아 끼운다."

라는 철학과 연결됩니다.

16. 이번 챕터를 사용자님의 창업 관점에서 보면

이 방법은 앞으로 ASP에서 고객의 업무를 AI 자동화하는 방법을 설계할 때도 그대로 사용할 수 있습니다.

예를 들어 어떤 회사의 업무가:

```
자료 수집
→ 고객 분석
→ 보고서 작성
→ 검수
→ 전달
```

이라면 처음부터 거대한 AI를 만들 필요가 없습니다.

먼저:

```
[수집]
↓
[분석]
↓
[작성]
↓
[검수]
↓
[전달]
```

이라는 뼈대를 만들고,

하나씩 실제 기능으로 바꾸면 됩니다.

이것이 사용자님이 앞으로 **AI 서비스 기획자·AI 컨설턴트**로 활동할 때 상당히 중요한 사고방식입니다.

★ 핵심 정리

이번 챕터의 핵심은 한 문장으로:

"부품을 완성한 뒤 조립하지 말고, 전체가 움직이는 최소한의 뼈대를 먼저 만든 다음 하나씩 완성하자."

입니다.

개발 순서

```
① 5개 Node 설계
↓
② State 정의
↓
③ 빈 Node 연결
↓
④ 전체 실행
↓
⑤ 수집 Node 완성
↓
⑥ 다시 전체 실행
↓
⑦ 선별 Node 완성
↓
⑧ 다시 전체 실행
```

이것만은 꼭 기억하세요

1. Node = 하나의 업무 단계
2. State = Node 사이를 이동하는 공용 작업 정보
3. State에는 단계 간에 전달해야 하는 정보만 넣는다.
4. Reducer = 여러 결과를 State에 어떻게 합칠지 정하는 규칙
5. 한 곳에서 새 값으로 교체해야 한다면 Reducer가 필요 없다.
6. 여러 워커의 결과나 여러 로그를 합쳐야 한다면 Reducer가 필요하다.
7. 빈 Node는 쓸모없는 임시 코드가 아니라 '걸어 다니는 뼈대'다.
8. 완벽하게 설계한 뒤 실행하는 것이 아니라, 실행하면서 조금씩 완성한다.

한 문장으로 정리

AI Agent 개발은 처음부터 완벽한 부품을 만드는 것이 아니라, State라는 공용 작업판 위에 전체 Workflow의 뼈대를 먼저 세우고, 각 Node를 하나씩 실제 기능으로 교체하면서 검증하는 과정입니다.

Chapter03

이번 챕터의 핵심은 "뉴스를 어떻게 가져올까?"라는 기술 문제가 아니라, 데이터를 가져오는 경로를 안정성·통제력·법적 위험까지 고려해 선택하는 방법입니다.

/강의정리 챕터 03

자료 수집 — 첫 번째로 정할 것

1. 이번 챕터에서 가장 먼저 배울 것

뉴스레터 에이전트를 만들 때 첫 번째 단계는 자료 수집입니다.

그런데 바로 코딩부터 하면 안 됩니다.

먼저 결정해야 합니다.

인터넷의 정보를 어떤 경로로 가져올 것인가?

정보를 가져오는 방법에 따라

- 안정성
- 데이터 품질
- 유지보수 비용
- 통제력
- 법적·정책적 위험

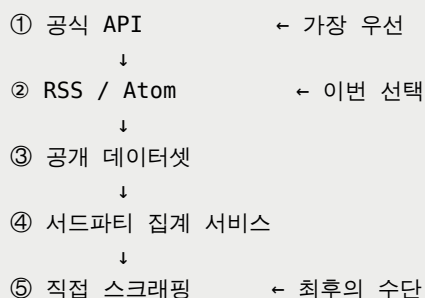
이 크게 달라집니다.

그래서 이번 강의에서는 **5단계 수집 경로 사다리**를 사용합니다.

2. 정보를 가져오는 5칸의 사다리

원칙은 아주 간단합니다.

위에서 가능한 방법이 있으면 아래로 내려가지 않는다.



위로 갈수록 일반적으로 공식성·안정성·예측 가능성이 높고,
아래로 갈수록 유지보수와 정책·법적 검토 부담이 커집니다.

3. ① 공식 API

가장 먼저 확인할 방법입니다.

API는 쉽게 말하면 서비스 제공자가 프로그램에게 정보를 가져가는 공식 창구를 만들어 놓은 것입니다.

예를 들어:

```
내 프로그램
↓
공식 API 요청
↓
JSON 데이터
```

처럼 정해진 형식으로 데이터를 받을 수 있습니다.

장점

- 공식적으로 제공되는 통로
- 데이터 형식이 비교적 명확
- 프로그램에서 처리하기 쉬움
- 제공자가 정한 사용 방법과 한도를 확인할 수 있음

단점

- API를 제공하지 않는 서비스가 많음
- API Key 등이 필요할 수 있음
- 호출 횟수 제한이 있을 수 있음
- 제공 데이터가 원하는 만큼 충분하지 않을 수 있음

따라서 **가능하다면 1순위**입니다.

4. ② RSS · Atom

이번 강의에서 선택한 방법입니다.

RSS는 언론사나 블로그가 새로운 글을 올렸다는 정보를 **프로그램이 읽을 수 있도록 공개하는 피드**입니다.

쉽게 말하면:

|"새 글이 나오면 이 목록에서 확인하세요."

라고 제공하는 자동 게시판과 비슷합니다.

```
언론사
↓
```

RSS



제목

링크

작성시간

짧은 설명



뉴스레터 에이전트

RSS의 장점

- 별도 인증 없이 접근 가능한 경우가 많음
- 오랫동안 사용되어 형식이 안정적
- 시간순으로 새 글을 가져오기에 적합
- 발행자가 공개적으로 제공하는 통로

단점

RSS에는 보통 **기사 전문이 아니라 제목과 짧은 설명 정도만** 들어 있습니다.

따라서 필요한 경우 원문을 별도로 확인해야 합니다.

또한 소스마다 날짜 형식이나 필드 구조가 조금씩 다를 수 있습니다.

5. 왜 뉴스에는 RSS가 잘 맞을까?

이번 프로젝트의 대상은 **뉴스**입니다.

뉴스는 본질적으로

|"새로운 글이 계속 추가되는 정보"

입니다.

RSS는 바로 이런 정보를 전달하기 위해 만들어졌습니다.

따라서

새 기사 발생



RSS에 추가



우리 프로그램이 확인



새 기사 수집

이라는 구조가 자연스럽습니다.

즉, 데이터의 성격과 수집 방식이 잘 맞습니다.

6. ③ 공개 데이터셋·아카이브

이미 누군가가 수집해서 만들어 놓은 데이터를 사용하는 방법입니다.

예를 들어 과거 10년간의 뉴스 데이터를 분석한다고 해보겠습니다.

이 경우 공개 데이터셋은 매우 유용합니다.

공개 데이터셋
↓
한꺼번에 다운로드
↓
과거 뉴스 분석

장점

- 대량의 데이터를 한꺼번에 확보
- 과거 데이터 분석에 유리
- 반복적으로 웹을 수집할 필요가 없음

단점

실시간성이 떨어집니다.

이번 프로젝트는

| "오늘 아침 최신 AI 뉴스"

가 목적이기 때문에 적합하지 않습니다.

7. ④ 서드파티 집계 서비스

여러 언론사의 뉴스를 이미 모아 놓은 서비스를 사용하는 방법입니다.

쉽게 말하면:

| "뉴스 수집은 귀찮으니까 돈을 내고 다른 회사에게 맡기자."

입니다.

장점

- 여러 소스를 한꺼번에 가져올 수 있음
- 직접 수집 시스템을 만들 필요가 적음
- 빠르게 서비스를 만들 수 있음

단점

가장 중요한 문제가 **통제력**입니다.

예를 들어 해당 서비스가

|"이 뉴스는 중요하지 않음."

이라고 판단해서 제외했다면 우리는 그 판단 과정에 개입하기 어렵습니다.

또한 서비스가 변경되거나 종료되면 우리 시스템도 영향을 받습니다.

따라서 **학습 목적과 직접 설계 경험을 중시하는 이번 강의에서는 선택하지 않습니다.**

8. ⑤ 직접 스크래핑

가장 아래에 있는 방법입니다.

웹사이트의 HTML을 직접 가져와 필요한 내용을 찾아내는 방식입니다.

```
웹페이지 HTML
↓
원하는 영역 찾기
↓
제목 / 가격 / 날짜 등 추출
```

예를 들어 웹페이지에서

```
<h1>새로운 AI 제품 출시</h1>
```

같은 부분을 찾아 제목을 가져오는 식입니다.

장점

공식 API나 RSS가 없어도 데이터를 가져올 수 있습니다.

그래서 정말 필요한 경우 강력합니다.

하지만 단점이 많습니다.

웹사이트 디자인이 바뀌면 코드가 깨질 수 있습니다.

예를 들어 기존에는:

```
<div class="article-title">
```

였는데 사이트 개편 후

```
<section class="headline">
```

로 바뀌면 기존 코드가 작동하지 않을 수 있습니다.

또한 사이트의 **robots.txt**, **이용약관**, **접근 정책** 및 **관련 법적·계약적 조건**을 확인해야 합니다.

따라서 가능한 경우에는 **공식 API**나 **RSS** 같은 공개된 통로를 먼저 사용하는 것이 좋습니다.

9. 왜 이번 프로젝트에서는 RSS를 선택했을까?

5가지 방법을 비교해보면 명확합니다.

방법	이번 프로젝트에서의 판단
① 공식 API	가장 좋지만 뉴스 매체에서 제공되지 않는 경우가 많음
② RSS/Atom	뉴스에 적합하고 공개 피드가 많음 → 선택
③ 데이터셋	실시간 뉴스에 부적합
④ 집계 서비스	편하지만 직접 판단·설계할 기회가 줄어들음
⑤ 스크래핑	유지보수 및 정책 검토 부담이 큼

따라서 기본적으로 **RSS/Atom**을 선택합니다.

단, 여기서 아주 중요한 포인트가 있습니다.

프로젝트 전체에서 하나의 수집 방법만 고르는 것이 아닙니다.

소스마다 가장 적절한 방법을 선택합니다.

10. Hacker News는 왜 API를 쓰는가?

예를 들어 Hacker News처럼 공개 API가 있다면 굳이 RSS만 고집할 필요가 없습니다.

API에서는 단순히 제목과 링크뿐 아니라

- 점수(points)
- 댓글 수(num_comments)

같은 정보도 얻을 수 있습니다.

이런 정보는 나중에

|"사람들이 실제로 관심을 많이 보인 뉴스인가?"

를 판단하는 데 활용할 수 있습니다.

즉,

```
TechCrunch → RSS
OpenAI → RSS
DeepMind → RSS
AI타임스 → RSS
Hacker News → API
```

처럼 **소스별로 최적의 방법을 선택**하는 것이 더 좋은 설계입니다.

11. 이번 챕터의 핵심 사고방식

이번 강의에서 정말 중요한 것은 RSS 자체가 아닙니다.

더 중요한 것은 **수집 경로를 선택하는 기준**입니다.

어떤 데이터를 가져오든 다음 순서로 생각하면 됩니다.

```
공식 API가 있는가?
↓
있으면 → API

없다면
↓
RSS / Atom이 있는가?
↓
있으면 → RSS

없다면
↓
공개 데이터셋으로 해결 가능한가?
↓
가능하면 → 데이터셋
```

없다면
↓
서드파티 서비스가 적절한가?
↓
그것도 어렵다면
↓
스크래핑 검토

즉,

| "어떻게든 가져온다"가 아니라 "가장 안전하고 안정적인 통로를 먼저 찾는다."

가 핵심입니다.

12. 객관식 퀴즈 정답

✓ 정답: ②

| 위 칸일수록 발행자가 '가져가도 된다'고 밝힌 통로여서 형식 안정성과 정책·법적 명확성을 함께 확보하기 쉽기 때문이다.

특히 여기서 중요한 것은 "위 칸 = 데이터가 더 많다"가 아니라는 것입니다.

공식 API가 RSS보다 반드시 데이터가 많은 것도 아닙니다.

핵심은:

공식성 + 안정성 + 예측 가능성 + 통제 가능한 사용 조건

입니다.

13. 주관식 퀴즈를 쉽게 생각해보기

질문은 경쟁사 제품 가격 변동 알림입니다.

뉴스와는 데이터의 성격이 다릅니다.

먼저 공식 API가 있다면 1칸을 선택할 수 있습니다.

하지만 API도 없고 RSS도 없고 공개 데이터셋도 실시간 가격을 제공하지 않는다면 5칸인 스크래핑을 검토해야 할 수 있습니다.

그때 포기하는 것은:

| 공식적이고 안정적인 데이터 접근 경로의 장점

입니다.

대신 새롭게 짚어지는 부담은:

- 웹페이지 구조 변경에 따른 유지보수
- 접근 제한
- robots.txt 및 이용약관 확인
- 데이터 추출 오류
- 정책 변화에 따른 대응

등입니다.

따라서 주관식 답변은 이렇게 정리하면 좋습니다.

경쟁사 가격 변동 알림에서는 공식 API나 피드가 없다면 5칸인 스크래핑으로 내려갈 수 있습니다. 대신 공식적인 데이터 제공 통로를 사용하는 안정성과 예측 가능성을 포기하고, 웹 페이지 구조 변경, 접근 정책, 이용약관 및 유지보수 문제를 직접 관리해야 합니다.

14. 고객님의 AI 컨설팅에 특히 중요한 부분

이 챕터는 앞으로 고객님의 AI 컨설팅을 할 때 상당히 중요한 질문을 하나 가르쳐 줍니다.

고객이

"이 데이터도 AI가 자동으로 가져오게 해주세요."

라고 했을 때 바로

"스크래핑하면 됩니다."

라고 하면 안 됩니다.

먼저:

- ① 공식 API?
↓
- ② RSS / 공개 피드?
↓
- ③ 공개 데이터?
↓
- ④ 외부 데이터 서비스?
↓
- ⑤ 정말 필요한 경우 스크래핑?

순서로 검토해야 합니다.

즉 AI 자동화에서 개발 능력만큼 중요한 것이 데이터 접근 경로를 설계하는 능력입니다.

이것은 앞으로 ASP에서 고객 업무를 분석할 때도 그대로 사용할 수 있는 사고방식입니다.

★ 이번 챕터 핵심 정리

정보 수집의 5단계 사다리

① 공식 API → ② RSS/Atom → ③ 공개 데이터셋 → ④ 서드파티 서비스 → ⑤ 직접 스크래핑

기본 원칙

| 위 칸에서 해결할 수 있으면 아래 칸으로 내려가지 않는다.

이번 프로젝트

| 뉴스라는 특성 때문에 RSS/Atom을 기본으로 선택

단, Hacker News처럼 공식 API가 있는 소스는 API를 사용합니다.

가장 중요한 이유

수집 방법은 단순한 코딩 문제가 아니라

데이터 품질 + 안정성 + 유지보수 + 통제력 + 정책·법적 위험

을 함께 결정하기 때문입니다.

🔥 이것만은 꼭 기억하세요

1. 데이터를 가져오기 전에 '어떤 경로가 가장 적절한가'부터 결정한다.
2. 공식 API가 있다면 가장 먼저 검토한다.
3. 뉴스처럼 지속적으로 업데이트되는 콘텐츠에는 RSS가 매우 유용하다.
4. RSS가 없다고 바로 스크래핑하지 말고 다른 대안을 먼저 확인한다.
5. 수집 방법은 프로젝트 전체에 하나를 고르는 것이 아니라 소스별로 선택할 수 있다.
6. Hacker News처럼 API가 제공되면 RSS보다 더 풍부한 신호를 얻을 수 있다.

7. 스크래핑은 강력하지만 유지보수와 접근 정책 검토 부담이 크므로 최후의 수단으로 생각한다.

한 문장으로 정리

AI 에이전트의 첫 단계인 데이터 수집은 "어떻게든 가져오는 것"이 아니라, 공식 API → RSS/Atom → 공개 데이터 → 외부 서비스 → 스크래핑 순으로 가장 안정적이고 적절한 데이터 접근 경로를 선택하는 설계 문제입니다.

Chapter04

이번 챕터는 “RSS를 선택했다”에서 한 단계 더 나아가, 실제로 어떤 RSS 소스를 믿고 사용할 것인지 검증하는 방법을 배우는 내용입니다.

핵심은 소스를 고를 때 감으로 판단하지 말고, 실제 데이터를 측정하고 관문(Gate)을 통과시킨다는 것입니다.

/강의정리 챕터 04

자료 수집 — 소스의 품질을 측정하고 관문을 세우기

1. 이번 챕터에서 배우는 것

챕터 03에서는 정보 수집 방법으로 RSS/Atom을 선택했습니다.

그런데 여기서 끝이 아닙니다.

같은 RSS라도 소스마다 품질이 다릅니다.

예를 들어:

A 사이트 → 1,000건 제공

B 사이트 → 10건 제공

A 사이트 → 긴 설명 제공

B 사이트 → 제목만 제공

A 사이트 → 오늘 기사 있음

B 사이트 → 일주일째 새 글 없음

따라서 중요한 질문은

|"RSS가 있느냐?"

가 아니라

|"이 RSS를 실제 뉴스레터 자동화에 사용할 수 있느냐?"

입니다.

2. 먼저 실제 데이터를 측정한다

이번 실습에서는 여러 RSS 소스에 실제 요청을 보내서 세 가지를 측정합니다.

① 한 번에 몇 건을 주는가?

1000건
500건
100건
10건

등 소스마다 다릅니다.

특히 **10건밖에 주지 않는 피드**는 주의해야 합니다.

하루에 해당 매체가 15개 기사를 발행한다면:

하루 기사 15개
↓
RSS는 최근 10개만 제공
↓
하루에 5개가 밀려날 가능성

이렇게 됩니다.

따라서 **RSS가 있다고 해서 무조건 하루치 뉴스를 모두 수집할 수 있는 것은 아닙니다.**

3. ② RSS가 제공하는 요약의 길이

두 번째는 RSS에 들어 있는 요약의 길이입니다.

대체로:

제목
+

짧은 설명

정도만 제공됩니다.

즉 RSS는 기본적으로 본문을 전달하기 위한 수단이라기보다 새 글이 올라왔음을 알려주는 피드입니다.

그래서 이번 시스템에서는:

```
RSS
↓
제목 + 링크 + 짧은 설명
↓
원문 페이지
↓
본문 추출
↓
AI 요약
```

이라는 구조를 사용합니다.

4. ③ 최신 글이 얼마나 최근 것인가?

세 번째는 최신 기사 날짜입니다.

예를 들어:

```
OpenAI → 오늘
DeepMind → 어제
어떤 블로그 → 7일 전
```

이라면 마지막 소스는 실시간 뉴스 수집에 적합하지 않을 수 있습니다.

특히 이번 시스템은 **최근 24시간 뉴스**를 가져옵니다.

따라서 오랫동안 새 글이 없는 소스는 매일:

```
0건
0건
0건
...
```

이 될 수 있습니다.

이것이 **G2 생존 관문**과 연결됩니다.

5. 1차 소스와 2차 소스

이번 챕터에서 중요한 개념입니다.

1차 소스

사건의 당사자가 직접 발표하는 정보입니다.

예:

- OpenAI 공식 발표
- DeepMind 공식 블로그
- Hugging Face 공식 발표

즉,

┆ "우리가 이런 일을 했다."

라고 당사자가 직접 이야기하는 것입니다.

2차 소스

당사자의 발표나 사건을 받아서 취재·분석하는 언론사입니다.

예:

- TechCrunch
- The Verge
- AI타임스

즉,

┆ "이 회사가 이런 일을 발표했다."

라고 전달하는 것입니다.

6. 왜 1차와 2차를 구분해야 할까?

아주 중요한 문제입니다.

예를 들어 OpenAI가 새로운 모델을 발표했다고 생각해보겠습니다.

그러면:

```
OpenAI 공식 발표
  ↓
TechCrunch 기사
  ↓
The Verge 기사
  ↓
AI타임스 기사
  ↓
전자신문 기사
```

처럼 **하나의 사건을 여러 언론사가 동시에 보도**합니다.

그런데 단순히 중요도 순으로 5개를 뽑으면:

```
1위 OpenAI 기사
2위 TechCrunch 기사
3위 The Verge 기사
4위 AI타임스 기사
5위 전자신문 기사
```

가 되어버릴 수 있습니다.

결국 **같은 사건으로 5개의 자리를 모두 차지하는 문제**가 발생합니다.

7. 그래서 **tier** 를 둔다

이를 해결하기 위해 소스에 **tier** 라는 정보를 붙입니다.

예:

```
OpenAI      → tier 1
DeepMind    → tier 1
TechCrunch  → tier 2
The Verge   → tier 2
AI타임스    → tier 2
```

그리고 **1차 소스는 경쟁에서 별도로 취급**할 수 있습니다.

즉:

1차 소스
→ 사실 자체로 중요

2차 소스
→ 여러 기사끼리 경쟁
→ 중요도 평가

이렇게 하면 정보의 성격에 맞게 선별할 수 있습니다.

8. 하지만 1차 소스라고 무조건 통과시키면 안 된다

여기에도 함정이 있습니다.

공식 블로그에도:

- 제품 발표
- 연구 결과

뿐 아니라

- 채용 공고
- 행사 안내
- 회사 홍보

등이 올라올 수 있습니다.

따라서

1차 소스 = 경쟁 면제

이지,

1차 소스 = 모든 검사 면제

가 아닙니다.

즉,

중요도 경쟁은 면제하되 품질 검사는 해야 합니다.

그리고 1차 소스가 너무 많으면 뉴스레터의 자리를 모두 차지할 수 있으므로 **상한(cap)**을 둘 수도 있습니다.

9. 이제 '관문(Gate)'을 만든다

이번 챕터의 가장 중요한 개념입니다.

소스를 평가할 때 모든 항목을 점수화할 필요는 없습니다.

먼저:

이 소스를 사용할 자격이 있는가?

부터 검사합니다.

이를 **관문(Gate)**이라고 합니다.

이번 강의에서는 세 가지 관문을 사용합니다.

G1 본문
↓
G2 생존
↓
G3 접근
↓
채택 가능

하나라도 통과하지 못하면 해당 소스를 사용하지 않습니다.

10. G1 — 본문 관문

질문

실제로 요약할 수 있는 본문을 확보할 수 있는가?

이것이 가장 중요합니다.

RSS의 짧은 설명만 가지고 AI에게

"이 기사를 자세히 요약해."

라고 하면 AI가 부족한 내용을 추측할 수 있습니다.

즉:

RSS
제목 + 짧은 설명
↓
AI

↓
"정보가 부족하니까 나머지는 추측..."

이런 상황이 생길 수 있습니다.

그래서 원문 페이지까지 접근하여 실제 본문을 추출합니다.

이번 실습에서는 **trafilatura**를 사용합니다.

RSS URL
↓
기사 URL
↓
원문 가져오기
↓
trafilatura
↓
본문 추출

11. RSS와 본문 추출은 별개의 단계다

이것이 이번 챕터에서 반드시 기억해야 할 부분입니다.

RSS가 제공하는 것은:

|"새로운 기사가 있습니다."

에 가깝습니다.

본문 추출은:

|"그 기사의 실제 내용을 가져옵니다."

입니다.

따라서:

[RSS 수집]
+
[원문 본문 추출]
↓
AI가 요약할 수 있는 자료

라는 구조가 필요합니다.

12. 왜 G1을 가장 중요하게 보는가?

본문이 없으면 AI가 제목만 보고 내용을 만들어낼 가능성이 높아집니다.

이것은 AI 시스템에서 매우 위험합니다.

예를 들어 제목이

|" 새로운 AI 모델 발표"

라고만 되어 있는데 AI에게 요약하라고 하면,

모델이 알고 있는 일반적인 지식을 섞어서 그럴듯한 설명을 만들어낼 수 있습니다.

그래서

|" 요약 품질의 문제처럼 보이지만 사실은 입력 데이터의 문제

일 수 있습니다.

이것이 바로 **Garbage In → Garbage Out**의 대표적인 사례입니다.

13. G2 — 생존 관문

질문

|" 최근에도 실제로 새로운 글이 올라오는가?

아무리 좋은 RSS라도 더 이상 업데이트되지 않는다면 의미가 없습니다.

예를 들어:

최근 14일 기사
→ 0건

이라면 죽은 소스일 가능성이 있습니다.

반대로:

최근 14일
→ 50건

이라면 살아 있는 소스라고 판단할 수 있습니다.

여기서 중요한 것은 **정답인 하나의 기준이 존재하는 것이 아니라는 점**입니다.

서비스 특성에 따라

| "14일 동안 최소 3건"

처럼 자신만의 임계값을 정하면 됩니다.

14. G3 — 접근 관문

질문

| **자동화 프로그램이 해당 사이트에 접근해도 되는가?**

기술적으로 접근할 수 있다고 해서 무조건 사용해도 되는 것은 아닙니다.

예를 들어:

403 Forbidden

이 반환되거나 사이트의 `robots.txt` 에서 자동 접근을 제한하고 있다면 해당 소스를 채택하지 않습니다.

이번 실습에서는 Python의

`urllib.robotparser`

를 활용하여 robots.txt의 자동 접근 정책을 확인하는 방법을 배웁니다.

15. 세 관문을 한 번에 정리하면

관문	질문	실패하면
G1 본문	요약할 충분한 원문이 있는가?	AI가 부족한 정보로 요약할 위험
G2 생존	최근에도 업데이트되는가?	매일 0건이 될 가능성
G3 접근	자동 수집이 허용되는가?	접근·정책 문제 발생

즉:

G1 본문

↓

G2 생존

↓

입니다.

16. 관문은 왜 '점수'보다 먼저인가?

이 부분도 중요합니다.

예를 들어 어떤 사이트가:

- 뉴스 품질 ★★★★★
- 업데이트 속도 ★★★★★
- 영향력 ★★★★★

이라고 해도 자동 수집 자체가 허용되지 않는다면 채택하면 안 됩니다.

따라서 먼저:

| 사용 가능한가?

를 검사하고,

그다음에:

| 사용 가능한 것 중 무엇이 더 좋은가?

를 평가합니다.

즉,

1단계

Gate = 자격 심사

2단계

Score = 경쟁 평가

라고 이해하면 쉽습니다.

17. 측정해야 하는 세 가지 운영 지표

이번 챕터에서는 나중에 실제 운영하면서 확인할 숫자도 미리 정합니다.

① 본문 추출 성공률

본문 추출 성공 기사
÷
본문 추출 시도 기사

예를 들어:

100건 시도
95건 성공
→ 성공률 0.95

갑자기 0.95에서 0.60으로 떨어진다면?

| 사이트 구조가 바뀌었거나 접근이 차단되었을 가능성
을 의심할 수 있습니다.

② 소스별 기여 건수

최종 뉴스 5개가 어디에서 왔는지 기록합니다.

OpenAI	1건
TechCrunch	2건
The Verge	0건
AI타임스	2건

어떤 소스가 계속 0건이라면 다시 평가해야 합니다.

③ 추출 소요 시간

본문을 가져오는 데 걸린 시간을 측정합니다.

예:

평균 1초
→ 평균 1초
→ 갑자기 평균 8초

그렇다면 특정 사이트가 느려졌거나 문제가 생겼을 가능성이 있습니다.

18. 왜 숫자를 기록해야 할까?

AI 자동화에서는 "뭔가 이상한 것 같다"는 느낌만으로 운영하면 안 됩니다.

예를 들어 요약이 갑자기 이상해졌다면:

```
요약 AI가 나빠졌나?  
↓  
본문 추출 성공률 확인  
↓  
95% → 62%  
↓  
본문 추출 문제가 원인
```

이라고 찾아낼 수 있습니다.

즉,

AI 결과의 품질을 직접 감으로만 평가하지 않고 앞 단계의 지표를 통해 원인을 추적한다.
는 것입니다.

19. 객관식 퀴즈 ① 정답

✓ 정답: ②

그 매체가 하루에 글을 몇 개 쓰는지 확인한다. 10건이 하루치보다 적다면 한 번 수집하는 것만으로 그날의 기사를 놓칠 수 있기 때문이다.

핵심은 RSS가 몇 건을 주는지 자체가 아니라, 그 숫자가 해당 매체의 발행량을 감당할 수 있는지입니다.

20. 객관식 퀴즈 ② 정답

✓ 정답: ②

본문이 없으면 언어 모델이 제목만 보고 내용을 지어낼 가능성이 커져 이후 품질 문제의 출발점이 되기 때문이다.

이것이 G1을 가장 중요한 관문으로 먼저 검사하는 이유입니다.

21. 이번 챕터에서 새롭게 배운 구조

챕터 03까지는:

```
뉴스 소스
↓
RSS
↓
수집
```

정도였다면,

이번 챕터에서는:

```
뉴스 소스
↓
RSS 확인
↓
소스 품질 측정
↓
G1 본문 검사
↓
G2 생존 검사
↓
G3 접근 검사
↓
채택
↓
수집
```

으로 발전했습니다.

즉 수집하기 전에 수집할 만한 소스인지 먼저 검사하는 것입니다.

22. 고객님의 AI 컨설팅 관점에서 중요한 부분

이 챕터는 고객님의 앞으로 AI 컨설팅을 할 때 상당히 중요합니다.

고객이

|"우리 회사 데이터를 AI가 자동으로 수집하게 해주세요."

라고 하면 단순히

|"API 연결하면 됩니다."

라고 답하는 것이 아니라 먼저 **데이터 소스 관문**을 만들어야 합니다.

```
G1 데이터 품질
G2 데이터 최신성
G3 접근 가능성
↓
사용 가능한 데이터인가?
↓
YES → 자동화
NO → 다른 방법 탐색
```

이렇게 접근하면 **AI를 단순히 사용하는 사람**이 아니라 **AI 업무 시스템을 설계하는 사람**의 사고방식을 갖게 됩니다.

★ 이번 챕터 핵심 정리

핵심 흐름

RSS 선택 → 실제 측정 → 소스 검증 → 관문 설정 → 채택

3가지 관문

- | **G1 본문** — 충분한 원문을 얻을 수 있는가?
- | **G2 생존** — 최근에도 계속 업데이트되는가?
- | **G3 접근** — 자동 수집이 허용되고 안정적으로 가능한가?

핵심 설계 원칙

- | 관문을 먼저 통과시키고, 통과한 소스끼리 우열을 평가한다.

🔥 이것만은 꼭 기억하세요

1. 같은 RSS라도 소스의 품질은 천차만별이다.
2. 소스를 감으로 선택하지 말고 실제 데이터를 측정한다.
3. RSS의 짧은 설명만으로 AI에게 요약시키지 않는다.
4. RSS 수집과 원문 본문 추출은 별개의 단계다.
5. G1(본문) → G2(생존) → G3(접근)의 관문을 둔다.

6. 1차 소스는 경쟁에서 별도로 취급할 수 있지만 검사까지 면제해서는 안 된다.
7. 운영에서는 성공률·기여 건수·처리 시간을 숫자로 기록한다.
8. AI 시스템에서는 '결과가 이상하다'보다 '어느 단계의 지표가 변했는가'를 찾아야 한다.

한 문장으로 정리

좋은 뉴스레터 에이전트는 RSS가 있다는 이유만으로 소스를 사용하는 것이 아니라, 실제 데이터를 측정된 뒤 본문·생존·접근이라는 3개의 관문을 통과한 소스만 채택하고 운영 지표로 계속 건강 상태를 확인하는 시스템입니다.

Chapter05

/강의정리 — 챕터 05. 수집 노드 만들기

이번 챕터에서는 앞에서 만들어 둔 빈 `collect` 노드에 실제 수집 기능을 넣습니다.

핵심은 아주 단순합니다.

여러 소스에서 가져오기 → 시간 범위에 맞게 걸러내기 → 중복 제거 → 실패한 소스는 기록하고 계속 진행하기

그리고 중요한 점은 `collect` 함수 하나만 수정한다는 것입니다.

앞 챕터에서 설계한 **State**와 **Graph** 구조는 그대로 유지합니다.

1. 이번 챕터의 목적

뉴스레터 에이전트의 첫 번째 단계인 **자료 수집**을 실제로 작동시키는 것입니다.

수집 노드는 다음 4가지 일을 합니다.

단계	하는 일	이유
① 수집	여러 RSS/API에서 기사 가져오기	뉴스 재료 확보
② 시간 필터	최근 <code>hours</code> 시간 이내 기사만 남김	오래된 기사 제거
③ 중복 제거	같은 URL의 기사 제거	같은 기사 반복 방지
④ 예외 처리	실패한 소스 기록 후 계속 진행	전체 파이프라인 중단 방지

즉,

100개 가져오기 → 24시간 이내 70개 → 중복 제거 후 60개

같은 흐름입니다.

2. 가장 중요한 개념 ① — 시간 창(Time Window)

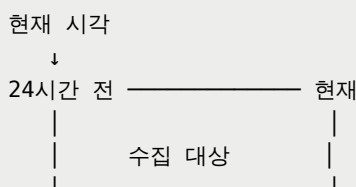
코드에서 핵심은 이것입니다.

```
if not at or at < cutoff
```

쉽게 말하면:

|"너무 오래된 뉴스나 날짜를 알 수 없는 뉴스는 버려라."

예를 들어 `hours = 24` 라면



24시간보다 오래된 기사는 제외합니다.

중요한 장점

시간 범위를 State의 `hours` 에서 가져오기 때문에 함수를 수정하지 않고도 범위를 바꿀 수 있습니다.

```
hours = 6
→ 최근 6시간 뉴스

hours = 24
→ 최근 24시간 뉴스

hours = 72
→ 최근 72시간 뉴스
```

즉, **로직과 설정을 분리**한 것입니다.

이것은 AI 에이전트를 만들 때 매우 중요한 설계 방식입니다.

| **코드는 그대로 두고 설정값만 바꿔서 행동을 변화시킨다.**

3. 가장 중요한 개념 ② — 중복 제거

다음 코드가 중복 제거의 핵심입니다.

```
key = e.link.split("?")[0]
```

왜 URL의 `?` 뒤를 잘라낼까요?

뉴스 사이트에서는 같은 기사에 이런 추적용 꼬리표가 붙는 경우가 많습니다.

```
https://example.com/news/123
```

```
https://example.com/news/123?utm_source=google
```

```
https://example.com/news/123?utm_source=newsletter
```

사실은 같은 기사인데 URL이 다르게 보입니다.

그래서

```
?utm_source=...
```

같은 추적 정보를 제거하고

```
https://example.com/news/123
```

만 비교합니다.

하지만 한계도 있습니다.

URL이 다르다면 같은 사건을 다룬 기사라도 제거하지 못합니다.

예를 들어:

```
TechCrunch → OpenAI 신제품 발표
```

```
The Verge → OpenAI 신제품 발표
```

```
AI타임스 → OpenAI 신제품 발표
```

세 기사는 서로 다른 URL이므로 모두 살아남습니다.

따라서 이번 단계의 중복 제거는

|"같은 URL의 중복" 제거

이지,

|"같은 사건에 대한 기사"의 중복 제거

가 아닙니다.

후자는 나중에 **AI 기반 중요도 선별/이벤트 클러스터링** 단계에서 다루는 것이 적절합니다.

4. 가장 중요한 개념 ③ — 예외 처리

이번 챕터에서 제가 가장 중요하다고 생각하는 부분입니다.

```
except Exception:
    dead.append(name)
```

예를 들어 뉴스 소스가 5개 있다고 해보겠습니다.

OpenAI	✓
DeepMind	✓
TechCrunch	✓
The Verge	✗ 서버 오류
AI타임스	✓

The Verge 하나가 실패했다고 전체 프로그램이 멈추면 안 됩니다.

따라서:

```
The Verge 실패
↓
dead에 기록
↓
다음 소스로 이동
↓
나머지 뉴스 계속 수집
```

하게 만듭니다.

5. ★ 이번 챕터의 핵심 — "조용한 실패"

여기서 자동화 시스템에서 아주 중요한 개념이 등장합니다.

조용한 실패(Silent Failure)

프로그램은 정상적으로 끝났는데 결과가 틀린 상태입니다.

예를 들어:

```
원래 뉴스 100개 수집
↓
한 소스가 실패
↓
실제로는 80개만 수집
↓
프로그램은 정상 종료
↓
아무도 모름
```

에러 메시지도 없습니다.

프로그램 입장에서는:

| "성공했습니다."

하지만 업무 입장에서는:

| "뉴스 20개를 놓쳤습니다."

입니다.

6. 왜 조용한 실패가 무서운가?

일반적인 오류는 발견하기 쉽습니다.

```
프로그램 실행
↓
❌ ERROR
↓
사람이 발견
↓
수정
```

하지만 조용한 실패는:

```
프로그램 실행
↓
✅ 정상 종료
↓
```

뉴스가 조금 부족함
↓
아무도 모름
↓
매일 반복
↓
몇 달 후 발견

이렇게 됩니다.

따라서 자동화 시스템에서는

❗ 실패하지 않는 것보다 실패했을 때 흔적을 남기는 것이 중요합니다.

7. `continue` 와 `except: pass` 가 위험한 이유

코딩을 하다 보면 이런 식으로 작성하기 쉽습니다.

```
continue
```

또는

```
except:  
    pass
```

뜻은 둘 다 대략

❗ "문제가 있지만 그냥 넘어가자."

입니다.

그런데 기록 없이 넘어가면 무슨 일이 있었는지 알 수 없습니다.

그래서 이번 코드에서는:

```
dead.append(name)
```

으로 실패한 소스의 이름을 기록합니다.

이것이 자동화 시스템에서 중요한 관찰 가능성(Observability)의 시작입니다.

8. 그렇다고 모든 것을 로그에 남기면 안 됩니다

여기에도 중요한 기준이 있습니다.

질문은 이것입니다.

|"이 실패가 최종 결과를 바꾸는가?"

예를 들어:

소스 하나 전체가 실패

OpenAI RSS ❌

→ 뉴스 전체가 빠짐

→ 반드시 기록

기사 하나의 날씨가 없음

기사 A → 날씨 없음

→ 그 기사 하나만 제외

→ 전체 결과에는 큰 영향 없음

→ 굳이 모든 것을 로그로 남길 필요는 없음

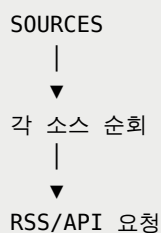
즉,

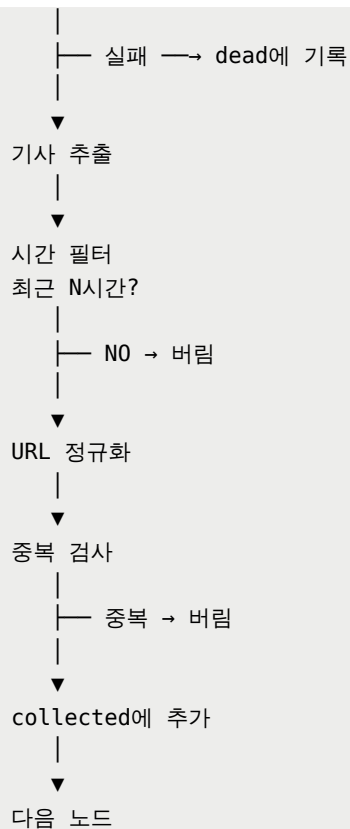
|"결과물을 바꾸는 중요한 실패는 기록한다."

이것이 로그 설계의 좋은 기준입니다.

9. 전체 collect 노드의 흐름

이번 챕터를 그림으로 보면 다음과 같습니다.





이것이 뉴스레터 에이전트의 첫 번째 실제 작업자(worker)입니다.

10. 앞 챕터의 State 설계가 왜 중요한가?

재미있는 부분이 있습니다.

이번 챕터에서는 **State**와 **Graph**를 수정하지 않았습니다.

앞 챕터에서 이미 이런 구조를 만들어 놓았기 때문입니다.

```

Graph
  ↓
collect
  ↓
select
  ↓
draft
  ↓
verify
  ↓
publish
  
```

처음에는 `collect` 가 빈 함수였습니다.

```
collect → 0개  
select → 0개  
draft → 0개  
...
```

이번에는 `collect` 함수만 실제 코드로 바꿉니다.

그러면 `build()` 가 이름을 보고 새로운 `collect` 함수를 연결합니다.

```
기존 Graph  
  |  
  ▼  
"collect"라는 이름을 찾음  
  |  
  ▼  
새 collect 함수 연결  
  |  
  ▼  
실제 뉴스 수집
```

이것이 좋은 설계인 이유

그래프 구조와 노드의 내부 구현이 분리되어 있기 때문입니다.

앞으로도:

```
collect 개선  
select 개선  
draft 개선  
verify 개선
```

을 각각 독립적으로 할 수 있습니다.

11. 🚀 더 해보기에서 배워야 할 것

실험 ① 소스 하나 추가하기

예를 들어 Hugging Face RSS를 추가합니다.

```
기존 소스  
→ 50개
```

Hugging Face 추가
→ 63개

그러면 알게 되는 것이 있습니다.

소스마다 뉴스 기여량이 다르다.

어떤 소스는 하루에 1개밖에 안 나오고, 어떤 소스는 20개가 나올 수도 있습니다.
따라서 나중에는

소스별 수집량
소스별 실패율
소스별 최신성
소스별 본문 추출 성공률

을 비교해야 합니다.

실험 ② hours 를 6 / 24 / 72로 바꾸기

예를 들어:

시간 범위	수집 건수
6시간	12
24시간	47
72시간	121

이렇게 차이가 날 수 있습니다.

여기서 중요한 것은 단순히

"72시간이 더 많이 나오니까 좋다."

가 아닙니다.

뉴스레터가 매일 발행된다면 72시간으로 가져오면 이전 뉴스가 다시 들어올 가능성이 있습니다.

반대로 6시간이면 중요한 기사를 놓칠 수도 있습니다.

즉,

시간 창은 뉴스레터 발행 주기와 연결해서 결정해야 합니다.

12. 이 챕터가 앞 챕터와 어떻게 연결되는가?

앞 챕터에서는

|"좋은 뉴스 소스를 어떻게 선택할 것인가?"

를 배웠습니다.

이번 챕터에서는

|"선택한 소스에서 실제로 어떻게 안전하게 가져올 것인가?"

를 배웁니다.

따라서 흐름은:

```
Chapter 03
수집 경로 선택
↓
Chapter 04
소스 품질 검증
↓
Chapter 05
실제 수집 노드 구현
↓
Chapter 06
중요 뉴스 선별
```

입니다.

즉, **설계** → **검증** → **구현** → **선별**의 순서입니다.

13. AI 개발자가 왜 이것을 알아야 할까요?

이 챕터는 단순한 RSS 코딩 수업이 아닙니다.

AI 에이전트를 실제 서비스로 만들 때 필요한 운영 원칙을 배우는 것입니다.

특히 다음 4가지는 꼭 기억할 필요가 있습니다.

① 실패를 전체 실패로 만들지 않는다

```
소스 A 실패
≠
전체 시스템 실패
```

가능하면 실패를 격리합니다.

② 실패의 흔적을 남긴다

실패 → dead 기록

그래야 나중에 원인을 찾을 수 있습니다.

③ 설정과 코드를 분리한다

hours = 24

같은 값을 State에서 관리하면 코드를 다시 작성하지 않고 행동을 바꿀 수 있습니다.

④ "프로그램이 돌아갔다"와 "업무가 제대로 됐다"는 다르다

이번 챕터는 앞에서 배운

실행 성공 vs 작업 성공

을 실제로 보여줍니다.

실행 성공
= 프로그램이 오류 없이 종료됨

작업 성공
= 필요한 뉴스가 제대로 수집됨

소스 하나가 몰래 빠졌는데 프로그램이 정상 종료됐다면?

실행 성공이지만 작업 성공은 아닙니다.

14. 💡 **사용자님의 ASP/MSM 사업과 연결하면**

이 개념은 앞으로 만드실 AI 컨설팅/콘텐츠 자동화 시스템에도 그대로 적용할 수 있습니다.

예를 들어 ASP에서 고객사의 AI 관련 자료를 자동 수집한다고 해보겠습니다.

고객사 홈페이지
블로그
뉴스
유튜브
SNS

보고서
↓
수집 Agent

이때 한 사이트가 접속 불가능하다고 전체 진단이 멈추면 안 됩니다.

A 사이트 ✖
B 사이트 ✔
C 사이트 ✔
D 사이트 ✔

그리고 결과에

A 사이트 수집 실패

라는 흔적이 남아 있어야 합니다.

그래야 고객에게

|"AI 분석 결과를 보니 문제가 없습니다."

라고 잘못 말하는 것을 막을 수 있습니다.

특히 **MSM처럼 사용자의 소중한 기억을 수집하는 시스템**에서도 같은 원칙이 중요합니다.

자료 수집
↓
일부 자료 실패
↓
조용히 누락 ✖

이 아니라

자료 수집
↓
일부 자료 실패
↓
실패 기록
↓
사용자에게 확인

이 되어야 합니다.

핵심 정리

이번 챕터에서 배운 것은 `collect` 라는 작은 함수 하나이지만, 실제로는 자동화 시스템의 기본 원리를 배웠습니다.

수집 노드의 4가지 역할

① 수집

→ 여러 소스에서 자료를 가져온다.

② 시간 필터

→ 정해진 시간 범위 밖의 자료를 제거한다.

③ 중복 제거

→ 같은 URL의 자료가 반복되는 것을 막는다.

④ 실패 격리 + 기록

→ 한 소스가 실패해도 나머지는 계속 진행하고 실패 사실을 기록한다.

★ 이것만은 꼭 기억하세요

1. `collect` 는 소스를 돌며 데이터를 모으는 첫 번째 작업 노드입니다.
2. `hours` 를 이용해 수집 시간 범위를 조절합니다.
3. URL의 추적 파라미터를 제거하면 같은 글의 단순 중복을 줄일 수 있습니다.
4. 한 소스의 실패가 전체 파이프라인의 실패가 되어서는 안 됩니다.
5. `except: pass` 처럼 조용히 삼키지 말고 중요한 실패는 기록해야 합니다.
6. 프로그램이 정상 종료됐다고 업무가 성공한 것은 아닙니다.
7. 이번 챕터의 핵심은 '수집'보다도 '안전하게 수집하는 방법'입니다.

한 문장으로 정리

뉴스레터의 `collect` 노드는 최근 자료만 골라 중복을 제거하고, 일부 소스가 실패해도 전체 수집은 계속되도록 만들어야 하며, 중요한 실패는 반드시 기록해야 합니다.



/강의정리 — 챕터 06. 중요한 소식 선정하기

이번 챕터에서는 수집한 100여 개의 뉴스 중에서 정말 중요한 5개를 어떻게 고를 것인가를 다룹니다.

핵심은 단순히 "기사마다 점수를 매기면 되지 않을까?"라는 생각에서 출발해서, 왜 그것보다 **상대평가 방식**이 더 적합한지를 이해하는 것입니다.

1. 이번 챕터의 핵심 질문

앞 챕터에서 뉴스가 이렇게 모였다고 해보겠습니다.

OpenAI	15개
TechCrunch	20개
The Verge	18개
DeepMind	12개
기타	40개
총	105개

그런데 뉴스레터에는 **딱 5개만** 넣고 싶습니다.

그러면 AI에게 이렇게 물어야 합니다.

┆ "각 기사가 몇 점인가?"

일까요?

아니면

┆ "오늘 이 105개 중 가장 중요한 5개는 무엇인가?"

일까요?

이번 챕터의 핵심은 바로 이것입니다.

┆ 문제가 '점수 매기기'가 아니라 '순위 정하기'라면, 상대평가가 더 적합합니다.

2. 처음 생각하기 쉬운 방법 — 독립 채점

가장 간단한 방법은 기사 하나씩 AI에게 점수를 달라고 하는 것입니다.

기사 A → 8점
기사 B → 7점
기사 C → 9점
기사 D → 7점
기사 E → 8점
...

그다음 점수가 높은 순서대로 5개를 뽑습니다.

처음에는 아주 합리적으로 보입니다.

하지만 문제가 발생합니다.

3. ! 점수가 생각보다 잘 퍼지지 않는다

12개 기사를 1~10점으로 평가하라고 했는데 실제로는:

점수	
10점	0개
9점	1개
8점	3개
7점	4개
6점	3개
5점	1개

처럼 **중간 점수에 몰릴 가능성이 높습니다.**

즉,

| "1~10점을 사용하라"고 했다고 해서 실제로 1~10점이 골고루 사용되는 것은 아닙니다.

4. 왜 이것이 문제인가?

우리는 **정확한 점수**가 필요한 것이 아닙니다.

우리의 요구사항은:

| **"5개를 골라 주세요."**

입니다.

그런데 이런 상황을 생각해 보세요.

1위 9점
2위 8점
3위 8점
4위 8점
5위 7점
6위 7점
7위 7점

5개를 골라야 하는데 5위와 6위가 동점입니다.

그러면 결국:

5위와 6위 중 누구를 선택할 것인가?

라는 새로운 문제가 생깁니다.

점수가 해결해주지 못합니다.

결국 **기사의 순서나 임의적인 기준**으로 잘라야 합니다.

5. 핵심: "채점 문제"가 아니라 "정렬 문제"

이것이 이번 챕터에서 가장 중요한 개념입니다.

절대평가

기사 하나를 보고:

| "이 기사는 10점 만점에 몇 점인가?"

를 판단합니다.

기사 A → 8점
기사 B → 7점
기사 C → 9점

상대평가

기사들을 한꺼번에 보고:

| "이 중 어떤 기사가 더 중요한가?"

를 판단합니다.

A vs B vs C vs D ...

↓

1위
2위
3위
4위
5위

뉴스레터의 요구사항은 상위 5개를 선택하는 것이므로 상대평가가 더 자연스럽습니다.

6. 네 가지 방법 비교

강의에서는 네 가지 방법을 비교합니다.

방법	방식	장점	단점
① 독립 채점	기사마다 점수 부여	간단, 병렬 처리 가능	동점, 비용 증가
② 토너먼트	기사끼리 1:1 비교	비교 판단이 정확	호출 횟수 폭증
③ 전체 비교	100개를 한 번에 비교	싸고 빠름	컨텍스트가 너무 길어짐
④ 2단 상대평가	예선 → 본선	비용과 품질의 균형	예선 탈락의 위험

이번 실습에서는 ④번을 선택합니다.

7. ① 독립 채점

기사 1 → 점수
기사 2 → 점수
기사 3 → 점수
...
기사 100 → 점수
↓
정렬
↓
TOP 5

장점

- 구조가 단순합니다.
- 각각 독립적으로 처리할 수 있습니다.
- 여러 기사를 동시에 처리하기 좋습니다.

단점

가장 큰 문제는 **동점**입니다.

그리고 100개라면 **100번 가까이 모델을 호출**해야 합니다.

즉,

| 기사 수가 많아질수록 비용도 증가합니다.

8. ② 토너먼트 방식

이번에는 기사 두 개를 놓고 묻습니다.

| "A와 B 중 어느 기사가 더 중요한가?"

```
A vs B → A 승
C vs D → D 승
E vs F → E 승
...
```

이렇게 계속 비교해서 순위를 만듭니다.

사람에게도 익숙한 판단 방식입니다.

예를 들어:

| "손흥민과 이강인 중 오늘 뉴스에서 누가 더 중요한가?"

처럼 두 대상을 비교하는 것은 점수를 직접 부여하는 것보다 쉬울 수 있습니다.

하지만 문제가 있습니다.

12개만 해도 모든 쌍을 비교하면:

$$12 \times 11 \div 2 = 66\text{번}$$

호출해야 합니다.

100개라면:

$$100 \times 99 \div 2 = 4,950\text{번}$$

입니다.

뉴스레터 하나 만들려고 4,950번 호출하는 것은 현실적으로 어렵습니다.

9. ③ 한 번에 전부 비교

이번에는 AI에게 100개를 한꺼번에 줍니다.

기사 1
기사 2
기사 3
...
기사 100

↓

"가장 중요한 5개를 골라라."

장점

호출 1번이면 됩니다.

그리고 모든 기사를 한꺼번에 비교하기 때문에 **상대적인 중요도 판단**도 가능합니다.

문제

100개의 제목과 설명을 한 번에 넣으면 너무 길어집니다.

그러면:

- 컨텍스트가 커짐
- 중간에 있는 기사를 놓칠 가능성
- 출력 형식이 깨질 가능성
- 처리 안정성 저하

등의 문제가 발생합니다.

10. ★ 우리가 선택한 방법 — 2단 상대평가

그래서 강의에서는 **예선** → **본선** 구조를 선택합니다.

100개 기사

↓

10개	10개	10개	...
-----	-----	-----	-----

↓

각 그룹에서 중요한 기사 선발

↓

약 20~30개
↓
본선
↓
최종 TOP 5

강의에서는 묶음당 **8개 정도**를 통과시키는 방식으로 예선 탈락 위험을 완화합니다.

핵심은:

전체를 한꺼번에 비교하기에는 너무 크고, 하나씩 채점하기에는 비효율적이므로 중간 크기의 그룹으로 나눠 상대평가한다.

입니다.

11. 왜 이것이 좋은 타협인가?

각 방법에는 포기해야 하는 것이 있습니다.

독립 채점
→ 간단하지만 동점 문제

토너먼트
→ 품질은 좋지만 너무 비쌌

전체 비교
→ 싸지만 컨텍스트 문제가 있음

2단 상대평가
→ 약간의 예선 손실을 감수하고
비용 + 비교 품질 + 컨텍스트 문제를 균형화

따라서 **완벽한 방법을 찾는 것이 아니라 요구사항에 맞는 방법을 선택하는 것이** 중요합니다.

이것은 AI 에이전트 설계에서 매우 중요한 사고방식입니다.

12. ⚠ 상대평가에도 문제가 있다 — "같은 사건" 중복

12개 기사를 한꺼번에 비교해보면 새로운 문제가 발견됩니다.

예를 들어:

- 1위 OpenAI 신제품 발표
- 2위 NVIDIA 신기술 발표
- 3위 OpenAI CEO 발언
- 4위 OpenAI 신제품 출시 관련 기사
- 5위 Google AI 발표

이렇게 될 수 있습니다.

3위와 4위가 사실상 **같은 사건**을 다루고 있을 수 있습니다.

그러면 뉴스레터 독자는 이렇게 생각합니다.

| "왜 비슷한 뉴스가 두 개나 들어갔지?"

13. 이것이 앞 chapter의 중복 제거와 연결된다

앞 chapter에서 URL 중복을 제거했습니다.

하지만 그것은:

| **같은 URL인가?**

만 판단합니다.

이번에는:

| **같은 사건인가?**

를 판단해야 합니다.

예를 들어:

A 매체
<https://a.com/123>

B 매체
<https://b.com/456>

URL은 다릅니다.

하지만 두 기사가 모두

| "OpenAI가 새로운 모델을 발표했다"

를 이야기한다면 **사건은 동일**합니다.

따라서 중복 제거는 두 단계로 생각할 수 있습니다.

1차: 코드 기반 중복 제거

같은 URL인가?

2차: 의미 기반 중복 제거

같은 사건인가?

두 번째는 AI가 판단하도록 할 수 있지만, **프롬프트에 명확하게 요구하지 않으면 AI가 반드시 알아서 처리해주는 것은 아닙니다.**

14. "AI가 알아서 알겠지"가 위험한 이유

모델 입장에서는 이렇게 생각할 수도 있습니다.

기사 A
"OpenAI, 새로운 AI 모델 공개"

기사 B
"OpenAI, 차세대 AI 모델 발표"

둘 다 중요한 뉴스입니다.

따라서 모델은:

└ "둘 다 중요하니까 둘 다 TOP 5에 넣자."

라고 판단할 수 있습니다.

우리가 원하는 것은:

└ "중요한 사건이 같다면 대표 기사 하나만 선택하자."

입니다.

그러므로 프롬프트에 **업무 규칙을 명시해야 합니다.**

예:

└ "같은 사건을 다룬 기사는 하나만 선정하세요."

15. 🧠 아주 중요한 설계 원칙

이번 챕터에서는 AI에게 질문을 정확하게 정의하는 것이 얼마나 중요한지 보여줍니다.

나쁜 질문

| "기사의 중요도를 1~10점으로 평가하세요."

더 좋은 질문

| "오늘 수집된 기사 중 가장 중요한 5개를 순위대로 선정하세요."

더 좋은 질문

| "오늘 수집된 기사 중 가장 중요한 5개를 선정하세요. 동일한 사건을 다룬 기사는 하나만 선택하세요."

즉,

AI의 성능을 높이는 것만큼 중요한 것이 AI에게 어떤 문제를 풀게 할 것인가를 정확히 정의하는 것입니다.

16. 🎯 후보 전체를 보는 것의 또 다른 장점

절대평가에서는:

기사 A → 8점

만 판단합니다.

그런데 상대평가에서는:

A
B
C
D
E
...

를 함께 보고

|"오늘은 엄청난 뉴스가 별로 없으니 이 기사가 3위 정도겠군."

이라고 판단할 수 있습니다.

즉, **점수 자체가 아니라 그날의 전체 상황 속에서 중요도를** 판단할 수 있습니다.

그래서 어제 1위였던 기사가 오늘은 6위가 될 수도 있습니다.

이것은 오류가 아닙니다.

후보군이 달라졌기 때문입니다.

17. 따라서 점수를 함부로 비교하면 안 된다

상대평가 결과는 **그날의 후보 구성에 따라 달라집니다.**

월요일
A → 1위

화요일
A → 6위

이것만 보고

|"AI가 월요일에는 A를 중요하다고 했는데 화요일에는 중요하지 않다고 했다."

라고 판단하면 안 됩니다.

월요일에는:

A가 가장 중요한 뉴스

였지만,

화요일에는:

A보다 훨씬 중요한 뉴스가 5개 등장

했을 수도 있기 때문입니다.

따라서 뉴스레터 시스템의 성능을 추적할 때는 **단순 점수보다 실제 선택 결과, 소스 분포, 수집량 등의 지표를 보는 것이 중요합니다.**

18. 🤖 AI 개발자에게 중요한 이유

이번 챕터는 사실 LLM을 어떻게 평가하고 의사결정에 활용할 것인가를 배우는 과정입니다.
AI 개발자가 기억해야 할 질문은 이것입니다.

| "내가 AI에게 무엇을 시키고 있는가?"

점수를 요구하면

AI는 점수를 만들려고 합니다.

순위를 요구하면

AI는 순위를 만들려고 합니다.

TOP 5를 요구하면

AI는 5개를 선택하려고 합니다.

따라서

| 출력 형식은 문제의 본질과 일치해야 합니다.

이것이 매우 중요합니다.

19. 💡 사용자님의 ASP / MSM과 연결하면

이 원리는 앞으로 만드실 AI 컨설팅 에이전트나 MSM에도 그대로 적용할 수 있습니다.

예를 들어 MSM에서 사용자가 100개의 기억 자료를 입력했다고 해보겠습니다.

AI에게:

| "각 기억의 중요도를 1~10점으로 평가하세요."

라고 하는 것보다,

| "이 사람의 인생 이야기를 구성할 때 반드시 포함되어야 할 핵심 사건 10개를 선정하세요."

라고 하는 것이 실제 목적에 더 가깝습니다.

또한:

| "같은 사건을 여러 번 언급한 자료는 하나의 사건으로 묶으세요."

라는 규칙도 필요합니다.

즉,

자료가 많다
↓
AI에게 점수만 매기게 한다 ✖
↓
실제 업무 목적을 정의한다
↓
상대적 비교/선별 기준을 설계한다
↓
최종 결과 개수를 명확히 한다

이런 사고방식이 **AI 서비스 기획자**에게 매우 중요합니다.

핵심 정리

이번 챕터의 가장 중요한 내용은 "100개 중 5개를 뽑는 문제를 어떻게 AI에게 시킬 것인가?"입니다.

① 독립 채점

기사별 점수 → 정렬

문제: 점수가 중간에 몰리고 동점이 발생합니다.

② 토너먼트

기사끼리 1:1 비교

장점: 상대적 판단이 좋음

문제: 호출 비용이 너무 큼

③ 전체 비교

100개를 한 번에 비교

장점: 싸고 상대적 판단 가능

문제: 컨텍스트가 너무 커짐

④ 2단 상대평가 ★

예선 → 본선

전체 후보를 적절한 크기로 나눠 비교한 후 최종 후보를 다시 비교합니다.

★ 이것만은 꼭 기억하세요

1. "몇 점인가?"와 "이 중 무엇이 더 중요한가?"는 서로 다른 질문입니다.
2. 고정된 개수의 결과를 뽑는 문제는 점수보다 순위/상대평가가 더 적합합니다.
3. 독립 채점은 점수가 물리면서 동점 문제가 발생합니다.
4. 토너먼트는 비교 품질이 좋지만 호출 비용이 너무 커질 수 있습니다.
5. 전체 비교는 싸지만 후보가 많아지면 컨텍스트 문제가 발생합니다.
6. 이번 강의에서는 예선 → 본선의 2단 상대평가를 선택했습니다.
7. URL이 다른 같은 사건의 기사는 별도의 의미적 중복 문제입니다.
8. AI가 알아서 같은 사건을 제거한다고 기대하지 말고 프롬프트에 명확한 규칙을 적어야 합니다.
9. AI 에이전트 설계에서는 "AI에게 무엇을 시킬 것인가?"를 정확히 정의하는 것이 매우 중요합니다.

🧠 한 문장으로 정리

뉴스 100개에서 5개를 골라야 한다면 개별 점수를 매기는 것보다 후보들을 상대적으로 비교해 순위를 정하는 것이 목적에 맞으며, 후보가 많을 때는 예선 → 본선의 2단 상대평가로 비용과 컨텍스트 문제를 균형 있게 해결할 수 있습니다.

Chapter07

📚 /강의정리 — 챕터 07. 후보가 수백 건이 되면

이번 챕터의 핵심은 뉴스 후보가 수백 건으로 늘어났을 때 LLM에게 어떻게 효율적으로 선별을 맡길 것인가입니다.

챕터 06에서는 12건 정도를 한 화면에 놓고 상대평가할 수 있었습니다. 하지만 실제 뉴스레터에서는 하루 수백 건이 들어올 수 있습니다.

따라서 이번에는 후보를 나눠서 예선하고 → 살아남은 후보만 본선에서 다시 비교합니다.

핵심 구조: 수백 건 → 묶음별 예선 → 예선 통과자 → 본선 → 최종 5건

1. 왜 후보를 나눠야 할까요?

12개 정도라면:

```
기사 1
기사 2
기사 3
...
기사 12
    ↓
LLM에게 한 번에 비교
    ↓
TOP 5
```

가 가능합니다.

하지만 300개라면:

```
기사 1
기사 2
...
기사 300
```

을 한꺼번에 보여줘야 합니다.

토큰 한도 안에 들어갈 수도 있지만 또 다른 문제가 생깁니다.

목록이 너무 길어지면 모델이 앞과 뒤는 잘 보면서 중간 내용을 상대적으로 놓칠 수 있습니다.

즉, 단순히 "토큰 한도에 들어가는가?"만 보면 안 됩니다.

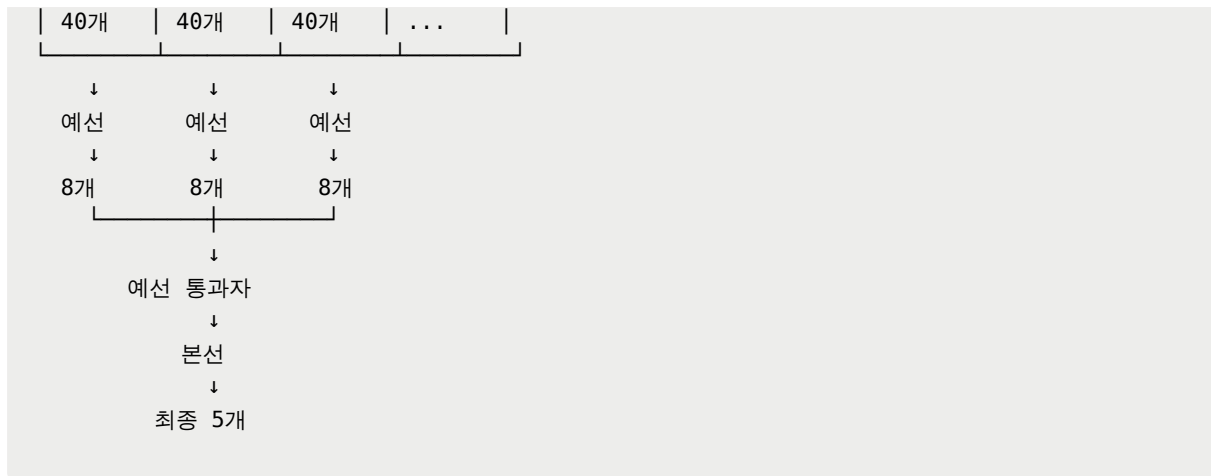
"모델이 모든 후보를 제대로 비교할 수 있는가?"도 봐야 합니다.

2. 해결책 — 예선 → 본선

이번 챕터에서는 후보를 여러 묶음으로 나눕니다.

예를 들어 40개씩 나누면:

```
300개 기사
    ↓
┌──────────┴──────────┐
```



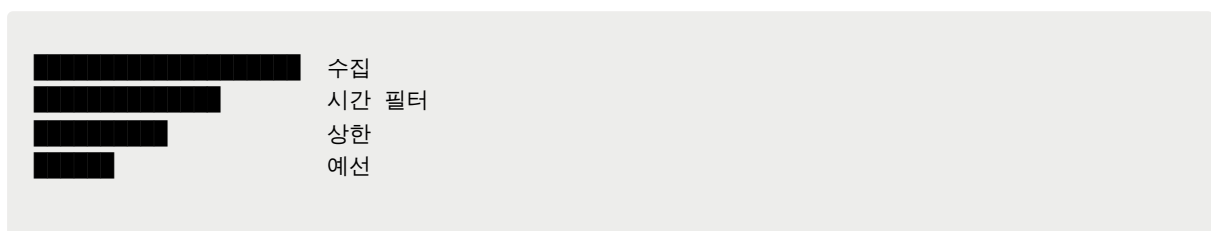
즉, 큰 문제를 작은 문제로 쪼개는 것입니다.

3. 전체 파이프라인에서 어디에서 줄어드는가?

이번 챕터에서는 뉴스가 단계마다 어떻게 줄어드는지를 관찰하는 것이 중요합니다.

- ① 수집
수백 건
↓
- ② 시간 창
24시간 이내
↓
- ③ 중복 제거
같은 URL 제거
↓
- ④ 소스별 상한
각 매체 최대 N건
↓
- ⑤ 예선
목록별 8건
↓
- ⑥ 본선
최대 5건
↓
- ⑦ 검수
합격 / 불합격

이것을 **깔때기(Funnel)**라고 생각하면 이해하기 쉽습니다.





중요한 것은 어디에서 몇 건이 빠지는지 숫자로 보는 것입니다.

4. 시간 창 → 중복 → 소스별 상한

특히 흥미로운 점이 있습니다.

시간 창

24시간 기준으로 자르면 오래된 기사가 대량으로 제거될 수 있습니다.

URL 중복

생각보다 많이 제거되지 않을 수 있습니다.

왜냐하면 앞 챕터에서 배웠듯이 URL 기반 중복 제거는

|"완전히 같은 주소인가?"

만 판단하기 때문입니다.

소스별 상한

오히려 이 단계에서 상당히 많은 기사가 줄어들 수 있습니다.

예를 들어:

TechCrunch → 50건
The Verge → 40건
기타 → 100건

이라고 해도 매체별 상한을 10건으로 두면:

TechCrunch → 10건
The Verge → 10건
기타 → ...

처럼 후보 수가 크게 줄어듭니다.

5. ★ 중요한 개념 — Sectioning

강의에서 소개하는 **sectioning(병렬화)**은 이번 챕터의 핵심 개념입니다.

쉽게 말하면:

└ **큰 일을 여러 조각으로 나눠 각각 처리한 다음 결과를 합치는 것**

입니다.

예를 들어 100명의 학생에게 시험지를 한꺼번에 채점하게 하는 대신:

1~25번 → A 선생님
26~50번 → B 선생님
51~75번 → C 선생님
76~100번 → D 선생님

처럼 나눠 처리하는 것입니다.

뉴스 선별에서도 똑같습니다.

100개
↓
25개 + 25개 + 25개 + 25개
↓
각 묶음 독립 처리
↓
결과 합치기

6. "서로 모르는 채로" 처리한다는 것

각 묶음의 예선은 서로의 존재를 모릅니다.

예를 들어:

묶음 A
40개 → 8개

묶음 B
40개 → 8개

A의 AI는 B의 기사들을 보지 않습니다.

이것은 단점처럼 보이지만 **큰 장점도 있습니다.**

장점 ① 코드가 단순합니다

묶음이 5개든 10개든 같은 코드를 사용할 수 있습니다.

장점 ② 병렬 처리가 가능합니다

서로 의존하지 않으므로 동시에 처리할 수 있습니다.

장점 ③ 하나가 실패해도 나머지는 살아 있습니다

A → 성공
B → 성공
C → ❌
D → 성공

C가 실패해도 A/B/D의 결과는 살아남습니다.

이것은 챕터 05에서 배운 **실패 격리**와 연결됩니다.

7. 하지만 치명적인 약점 — "묶음 운"

예선의 가장 큰 문제입니다.

예를 들어:

묶음 A
1위
2위
...
8위 ← 통과
9위 ← 탈락

그런데 A의 9위가 사실은 아주 좋은 기사일 수도 있습니다.

반면:

묶음 B
3위 ← 통과

가 상대적으로 평범한 기사일 수도 있습니다.

즉,

어떤 기사와 같은 묶음에 들어갔느냐에 따라 결과가 달라질 수 있습니다.

이것을 강의에서는 **묶음 운**이라고 표현합니다.

8. 묶음 운을 어떻게 줄일까요?

최종적으로 5개만 필요하다고 해서 예선에서 묶음마다 5개만 뽑으면 안 됩니다.

예를 들어:

40개
↓
5개

라고 하면 너무 많은 좋은 후보가 탈락할 수 있습니다.

그래서:

40개
↓
8개

정도로 **넉넉하게 통과**시킵니다.

그러면 본선에 더 많은 후보가 올라갑니다.

40개 × 여러 묶음
↓
묶음별 8개
↓
예선 통과 후보
↓
본선에서 다시 상대평가
↓
TOP 5

즉,

예선은 완벽한 최종 선발이 아니라 좋은 후보를 놓치지 않는 것이 목적입니다.

이것이 매우 중요합니다.

9. 예선과 본선의 역할을 다르게 생각하세요

예선

"좋은 후보를 최대한 살려두자."

목적 → 후보 압축

본선

"살아남은 후보 중 정말 중요한 5개를 고르자."

목적 → 최종 순위 결정

따라서 두 단계는 같은 일을 하는 것이 아닙니다.

예선 = Recall 중심

본선 = Precision 중심

초보자 입장에서는 어렵게 생각하지 않고:

예선은 좋은 것을 놓치지 않는 단계, 본선은 정말 좋은 것만 고르는 단계

라고 이해하시면 됩니다.

10. 구조화 출력(Structured Output)

이번 챕터에서 또 하나의 매우 중요한 개념이 나옵니다.

구조화 출력(Structured Outputs)입니다.

챕터 06에서는 AI에게:

"번호만 심표로 출력하세요."

라고 부탁했습니다.

예를 들어:

3, 7, 9, 12, 18

을 기대한 것입니다.

그런데 AI가 가끔:

제가 선정한 기사는 다음과 같습니다.

3번, 7번, 9번, 12번, 18번입니다.

라고 할 수 있습니다.

그러면 프로그램에서

```
int(x)
```

같은 처리를 할 때 오류가 발생할 수 있습니다.

11. 구조화 출력은 무엇인가?

쉽게 말하면:

AI가 자유롭게 글을 쓰지 못하고, 우리가 정한 양식에 맞춰 답하도록 만드는 것입니다.

예를 들어 다음과 같은 형태를 요구합니다.

```
Pick
├─ number
├─ event
└─ reason

Shortlist
└─ picks
```

그러면 AI가 정해진 구조에 맞춰 결과를 반환합니다.

12. 자유 문장 vs 구조화 출력

자유 문장

3번이 좋습니다.
이유는...
그리고 7번도 중요합니다.

사람에게는 읽기 좋습니다.

하지만 프로그램이 처리하기 어렵습니다.

구조화 출력

```
{  
  "number": 3,  
  "event": "OpenAI 신제품 발표",  
  "reason": "시장 영향력이 큼"  
}
```

사람에게는 조금 딱딱하지만 프로그램이 처리하기 매우 좋습니다.

즉,

사람에게 보여주는 출력과 프로그램이 다음 단계에서 사용하는 출력은 다르게 설계해야 합니다.

13. 구조화 출력의 중요한 한계

여기서 아주 중요한 구분이 있습니다.

구조는 보장할 수 있습니다.

예:

```
event 필드가 존재한다  
number 필드가 존재한다  
reason 필드가 존재한다
```

하지만 내용의 정확성까지 보장하지는 않습니다.

예를 들어:

```
기사 A → event = "OpenAI 발표"  
기사 B → event = "OpenAI 신제품"
```

이라고 하면 실제로는 같은 사건인데 서로 다른 라벨을 붙일 수도 있습니다.

즉,

스키마는 형식을 강제하지만 판단 자체를 강제하지는 않습니다.

이것은 AI 시스템을 설계할 때 매우 중요한 차이입니다.

14. 프롬프트와 Hard Constraint의 차이

이 부분은 앞에서 배우신 **instruction vs hard constraint**와도 연결됩니다.

프롬프트

"같은 사건이면 같은 라벨을 사용하세요."

→ AI에게 요청하는 규칙

코드

```
if event in seen_events:
    continue
```

→ 프로그램이 강제로 막는 규칙

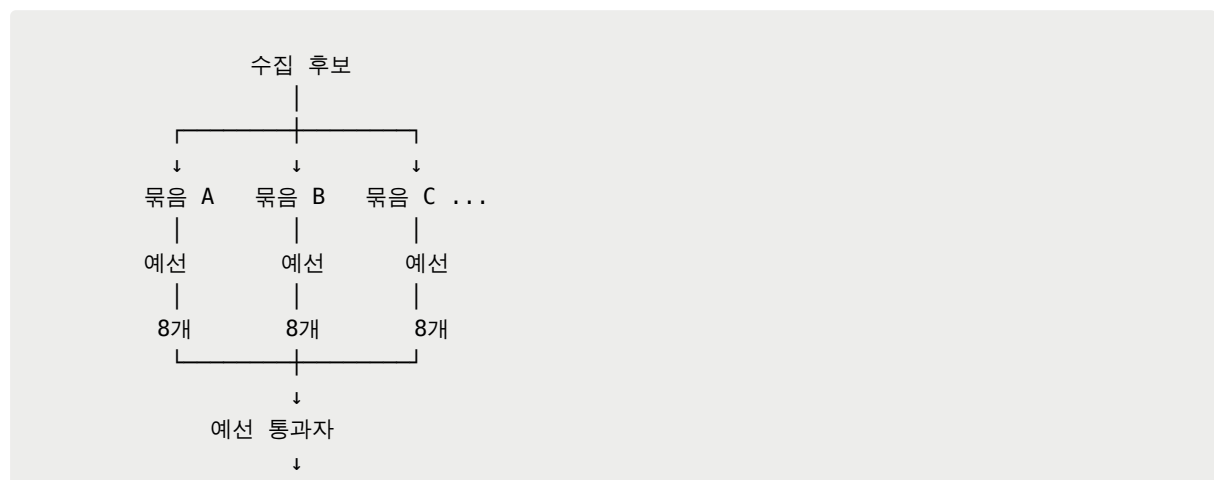
즉,

AI에게 부탁하는 것과 프로그램이 강제로 검사하는 것은 다릅니다.

강의에서 계속 강조하는 이유가 바로 이것입니다.

15. 예선 → 본선 전체 구조

이제 전체 **select** 노드를 보면 훨씬 이해하기 쉽습니다.



본선 상대평가
↓
TOP 5 선정

그리고 이 전체 과정을 하나의 `select` 노드에서 수행합니다.

16. 이 구조를 평가하려면 무엇을 봐야 할까요?

단순히

| "최종 5개가 마음에 든다."

만 보면 안 됩니다.

강의에서는 세 가지 지표를 봅니다.

① 묶음별 통과선

각 묶음의:

8등 → 마지막 통과
9등 → 첫 탈락

을 봅니다.

묶음마다 8등 수준이 크게 다르다면 묶음 운이 크다는 의미입니다.

② 본선 탈락 이유

최종 5개에 들어가지 못한 기사에:

중복
영향력 부족
이미 다룬 사건
정보 가치 부족

등의 이유를 남깁니다.

이것이 중요한 이유는:

| 탈락한 좋은 기사가 계속 쌓인다면 예선 구조에 문제가 있다는 신호이기 때문입니다.

즉, 최종 결과만 보지 말고 탈락 결과도 봐야 합니다.

③ 예선 통과 → 최종 발행 비율

예를 들어:

예선 통과 → 15개
최종 발행 → 5개

라면:

$$5 \div 15 = \text{약 } 33\%$$

입니다.

이 비율이:

너무 높다면

예선 15개
↓
최종 5개

- 본선에서 고를 여지가 적음
- 예선을 너무 빡빡하게 했을 가능성

너무 낮다면

예선 50개
↓
최종 5개

- 본선이 너무 많은 후보를 처리해야 함
- 예선이 너무 느슨할 가능성

따라서 예선은 적당한 후보 수를 남겨야 합니다.

17. 🚨 중요한 운영 원칙 — 최종 결과만 보지 마세요

자동화 시스템에서 아주 중요한 사고방식입니다.
뉴스레터가 이상하게 재미없어졌다고 해보겠습니다.
최종 결과만 보면:

|"요즘 AI 업계에 큰 뉴스가 없나?"

라고 생각할 수 있습니다.
하지만 실제로는:

수집 → 정상
시간 필터 → 정상
예선 → 좋은 기사 탈락
본선 → 정상

일 수도 있습니다.
따라서 **탈락자와 탈락 이유를 확인해야 합니다.**
이것은 앞에서 배운 **조용한 실패**와 연결됩니다.

18. AI 개발자에게 왜 중요한가?

이번 챕터에서는 AI 에이전트를 만들 때 필요한 핵심적인 **확장성(Scalability)** 사고를 배웁니다.

처음에는:

12개 → LLM 한 번

으로 해결할 수 있습니다.
하지만 데이터가:

12
→ 100
→ 300
→ 1,000

으로 증가하면 같은 방법을 계속 사용할 수 없습니다.
따라서 AI 시스템은 처음부터:

| "데이터가 10배가 되면 어떻게 되는가?"

를 생각해야 합니다.

19. 💡 사용자님의 ASP / MSM과 연결하면

이 개념은 사용자님이 만들려는 **MSM(My-Story-Maker)**에도 그대로 적용할 수 있습니다.

예를 들어 한 사람이 자신의 인생과 관련된 자료를:

일기 200개
SNS 500개
사진 1,000장
음성 50개

가지고 있다고 생각해보겠습니다.

이것을 AI에게 한 번에 모두 넣는 것은 좋은 방법이 아닙니다.

대신:

전체 자료
↓
주제별/시기별 묶음
↓
각 묶음에서 핵심 기억 선별
↓
핵심 기억 통합
↓
전체 인생 이야기의 핵심 사건 선정

으로 만들 수 있습니다.

뉴스레터의:

| 예선 → 본선

구조가 MSM의:

| 자료별 선별 → 전체 스토리 선별

로 그대로 확장될 수 있는 것입니다.

이런 구조를 이해해두시면 앞으로 AI 서비스 기획자로서 대량 데이터를 다루는 에이전트 설계를 훨씬 잘 이해하실 수 있습니다.

핵심 정리

이번 챕터는 크게 세 가지를 배운 것으로 정리할 수 있습니다.

① 대량 후보는 나눠서 처리한다

수백 건을 한꺼번에 LLM에게 넣지 않고:

묶음 → 예선 → 통과자 통합 → 본선

으로 처리합니다.

② 예선에는 "묶음 운"이 존재한다

서로 다른 묶음의 기사끼리 비교하지 못하기 때문에 좋은 기사가 탈락할 수 있습니다.

그래서 예선 통과 수를 넉넉하게 잡아 손실을 줄입니다.

③ 구조화 출력으로 AI의 출력 형식을 안정화한다

자유 문장을 파싱하는 대신 Pydantic 등의 스키마를 사용하여 프로그램이 사용할 수 있는 구조로 받습니다.

단,

구조는 강제할 수 있지만 판단의 정확성까지 보장하는 것은 아닙니다.

★ 이것만은 꼭 기억하세요

1. 후보가 수백 건이 되면 한 번에 비교하는 것보다 나눠서 처리하는 것이 안정적입니다.
2. 예선은 좋은 후보를 최대한 살려두는 단계입니다.
3. 본선은 살아남은 후보 중 최종 5개를 상대적으로 비교하는 단계입니다.
4. 묶음으로 나누면 '묶음 운'이라는 새로운 문제가 생깁니다.
5. 묶음 운을 줄이기 위해 최종 개수보다 훨씬 많은 후보를 예선에서 통과시킵니다.
6. Sectioning은 큰 입력을 여러 부분으로 나눠 독립적으로 처리하고 결과를 합치는 병렬화 패턴입니다.
7. 구조화 출력은 AI의 출력 형식을 안정적으로 만들어 줍니다.

8. 하지만 구조화 출력이 내용의 정확성까지 보장하는 것은 아닙니다.
9. 프롬프트로 부탁하는 규칙과 코드로 강제하는 규칙은 구분해야 합니다.
10. AI 에이전트의 품질은 최종 결과만 보지 말고 '어떤 후보가 왜 탈락했는가'까지 추적해야 합니다.

한 문장으로 정리

후보가 수백 건으로 늘어나면 입력을 묶음으로 나눠 예선에서 넉넉하게 후보를 살리고, 통과자만 본선에서 상대평가하며, 구조화 출력과 탈락 이유·통과율 같은 지표를 활용해 선별 과정 자체를 검증해야 합니다.

Chapter08

/강의정리 — 챕터 08. 제목과 링크만 있는 상태

이번 챕터에서는 지금까지 제목과 링크만 가지고 있던 뉴스 후보에 실제 기사 본문을 붙이고, 이를 읽을 만한 뉴스레터 콘텐츠로 만드는 과정을 배웁니다.

핵심은 두 가지입니다.

- ① 기사마다 독립적으로 취재·요약한다.
- ② 나중에 자동 검수하려면 처음부터 결과를 검증 가능한 구조로 만들어야 한다.

그리고 LangGraph의 중요한 기능인 ** `Send` 를 이용한 Fan-out(분기 처리)**도 등장합니다.

1. 현재 뉴스레터의 문제

앞 단계까지의 상태를 보면:

수집
↓
선별
↓
① 제목
② 링크

정도만 있습니다.

따라서 최종 결과도:

- ① OpenAI 새로운 AI 모델 발표
https://...
- ② NVIDIA 새로운 칩 공개
https://...
- ③ Google AI 업데이트
https://...

처럼 링크 목록에 가깝습니다.

뉴스레터라면 이것만으로는 부족합니다.

독자는 링크를 하나씩 눌러서 직접 기사를 읽어야 하기 때문입니다.

따라서 이번에는:

제목 + 링크
↓
기사 본문 가져오기
↓
본문 읽기
↓
헤드라인 + 요약 + 중요성

으로 바꿉니다.

2. G1 관문과 연결하기

챕터 04에서 배웠던 **G1 본문 관문**을 기억하시면 좋습니다.

|"요약할 만큼 원문 내용이 확보되어 있는가?"

본문이 없다면 AI는:

기사 제목
↓
AI가 추측
↓
그럴듯한 요약 생성

을 할 위험이 있습니다.

이것이 바로 **환각(Hallucination)**의 중요한 원인입니다.

이번 챕터에서 실제 본문을 가져오는 이유가 바로 이것입니다.

```
RSS
→ 제목/링크
↓
본문 추출
↓
LLM 요약
```

즉,

G1에서 확인했던 '재료가 있어야 한다'는 원칙을 실제 코드로 구현하는 단계입니다.

3. ★ 먼저 결정해야 할 것 — 요약을 어떤 구조로 받을 것인가?

" 기사를 요약해 주세요."

라고 하면 AI는 보통 이런 식으로 답합니다.

```
OpenAI가 새로운 AI 기술을 발표했습니다.
이번 기술은 ...
업계에서는 ...
```

사람이 읽기에는 괜찮습니다.

하지만 **자동화 시스템에서는 문제가 있습니다.**

왜냐하면 이 문단에서:

```
사실
해석
의견
```

이 어디에 있는지 프로그램이 알기 어렵기 때문입니다.

그래서 이번에는 결과를 세 칸으로 나눕니다.

4. 세 가지 칸으로 나누기

이번 챕터의 기본 구조는:

```
headline
summary
why
```

입니다.

① **headline**

뉴스 카드에 들어갈 **헤드라인**

② **summary**

기사의 핵심 내용을 담은 **세 문장 요약**

③ **why**

"**왜 중요한가?**"에 대한 한 문장

예를 들면:

```
headline
→ OpenAI, 새로운 AI 모델 공개

summary
→ OpenAI가 새로운 모델을 공개했습니다.
→ 기존 모델보다 ...
→ 개발자들은 ...

why
→ 국내 개발팀이 향후 AI 서비스 전략을 세울 때 참고할 만한 변화입니다.
```

5. 왜 굳이 세 칸으로 나눌까요?

가장 중요한 이유는 **검증 방법이 다르기** 때문입니다.

headline 과 **summary**

원문에 있는 내용을 요약한 것입니다.

따라서:

원문

↓

headline / summary

를 비교할 수 있습니다.

why

조금 다릅니다.

|"국내 개발팀에게 왜 중요한가?"

라는 **독자 관점의 해석**입니다.

원문에 그 문장이 그대로 존재하지 않을 수 있습니다.

따라서:

원문 ↔ summary

처럼 검사할 수 있지만

원문 ↔ why

는 같은 방식으로 검사하기 어렵습니다.

6. 💡 이것이 이번 챕터의 가장 중요한 개념입니다

|"출력 스키마를 정하는 것은 단순히 출력 모양을 예쁘게 만드는 일이 아닙니다.

실제로는:

|"나중에 무엇을 자동으로 검사할 수 있는가?"를 미리 결정하는 일입니다.

예를 들어:

문단 하나

로 받으면 프로그램이:

어디까지가 사실인가?
어디부터가 해석인가?
무엇을 다시 작성해야 하는가?

를 알기 어렵습니다.

반면:

headline
summary
why

로 나누면 각각 다른 검사를 적용할 수 있습니다.

7. 네 가지 출력 설계 방법

강의에서는 네 가지 방법을 비교합니다.

방법	특징	장점	단점
① 자유 텍스트	문단 하나	자유롭고 간단	자동 검증 어려움
② 검증별 칸 분리 ★	headline/summary/why	검증·재작성 쉬움	필드가 늘면 비용 증가
③ 근거 구절 포함	요약+원문 근거	검증 강력	출력 길고 비용 증가
④ 단계 분리	요약→해석→분류	각 단계 단순	호출 횟수 증가

이번에는 ②번을 선택합니다.

8. 왜 ③과 ④를 선택하지 않았을까요?

③ 근거 구절까지 받기

예:

요약 문장
→ 원문 근거 문장

을 함께 받습니다.

검증하기는 좋지만 출력이 길어지고 비용이 증가합니다.

또한 AI가 근거 구절 자체를 잘못 만들어낼 가능성도 있습니다.

④ 여러 번 호출

예:

```
1차 → 요약  
2차 → 중요성  
3차 → 토픽
```

으로 나눌 수 있습니다.

각 프롬프트가 단순해진다는 장점이 있습니다.

하지만 기사 5개라면 호출이:

```
5 × 3 = 15번
```

으로 늘어날 수 있습니다.

따라서 현재 뉴스레터 구조에서는 과합니다.

9. 구조화 출력(Structured Output)

여기서 챕터 07에서 배운 구조화 출력이 다시 등장합니다.

Pydantic 모델 등을 이용하여 AI에게:

```
headline  
summary  
why
```

라는 구조를 미리 알려줍니다.

그러면 AI의 답을 프로그램이 안정적으로 사용할 수 있습니다.

10. ⚠ 하지만 스키마가 모든 것을 보장하지는 않습니다

이것이 아주 중요합니다.

예를 들어 스키마에:

| **headline** 은 한국어로 작성하세요.

라고 적었다고 해보겠습니다.

그런데 영어 기사를 넣으면 AI가:

OpenAI launches new AI model...

처럼 영어로 답할 수도 있습니다.

즉,

스키마가 보장하는 것

headline 필드가 있다
summary 필드가 있다
why 필드가 있다

스키마가 보장하지 않는 것

한국어인가?
정확한 내용인가?
환각이 없는가?
세 문장인가?

따라서:

| **형식은 구조화 출력으로 통제하고, 내용은 별도의 검증으로 확인해야 합니다.**

이 원칙을 꼭 기억하셔야 합니다.

11. 영어 기사 → 한국어 요약 문제

실습에서 흥미로운 문제가 발생합니다.

한국어 뉴스레터를 만들라고 여러 번 지시했는데도 영어로 출력될 수 있습니다.

왜일까요?

LLM은 일반적으로 **입력의 언어를 따라가려는 강한 경향**이 있기 때문입니다.

```
영어 기사
↓
LLM
↓
영어 요약
```

이것이 프롬프트보다 강하게 작동할 수 있습니다.

따라서 단순히:

|"한국어로 써주세요."

라고 부탁하는 것만으로 충분하지 않습니다.

12. 프롬프트 → 검사 → 재시도

이 문제의 좋은 해결 방식은:

```
LLM
↓
한국어 요약
↓
코드 검사
↓
한국어인가?
├ YES → 통과
└ NO → 재요청
```

입니다.

예를 들어 코드로 한글 포함 여부를 검사할 수 있습니다.

```
re.search(r'[가-힣]', out.summary)
```

한글이 없다면:

|"반드시 한국어로 다시 작성하세요."

라고 재요청할 수 있습니다.

이것은 앞에서 배운 원칙과 정확히 연결됩니다.

| 프롬프트는 요청이고, 코드는 검사입니다.

13. ★ 이제 중요한 LangGraph 개념 — Send

이번 챕터에서 LangGraph의 중요한 기능이 등장합니다.

Send

입니다.

왜 필요할까요?

앞에서는 `select` 가 최종 후보 5개를 만들었습니다.

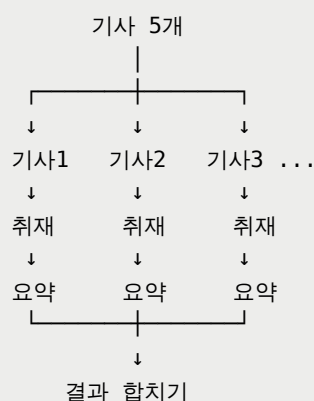
이제 각각의 기사를 취재해야 합니다.

기사 1 → 본문 추출 → 요약
기사 2 → 본문 추출 → 요약
기사 3 → 본문 추출 → 요약
기사 4 → 본문 추출 → 요약
기사 5 → 본문 추출 → 요약

입니다.

14. Fan-out — 일을 여러 갈래로 펼치기

이것을 **Fan-out**이라고 생각하면 쉽습니다.



한 사람이 5개의 일을 순서대로 하는 것이 아니라:

| 5개의 작업자를 동시에 출발시키는 것

과 비슷합니다.

15. **Send** 가 하는 일

조건부 엣지에서 일반적으로:

```
"다음 노드 이름"
```

을 반환할 수도 있습니다.

그런데 **Send** 리스트를 반환하면:

```
Send(노드, 기사1)  
Send(노드, 기사2)  
Send(노드, 기사3)  
Send(노드, 기사4)  
Send(노드, 기사5)
```

처럼 기사 수만큼 작업을 펼칠 수 있습니다.

따라서 기사 수가:

```
3개 → 3개 워커  
5개 → 5개 워커  
10개 → 10개 워커
```

처럼 동적으로 바뀝니다.

16. 워커가 받는 것은 전체 State가 아니다

이 부분도 중요합니다.

Send 로 실행되는 워커는 전체 State를 받는 것이 아니라:

```
{"item": 기사 하나}
```

를 받습니다.

즉:

| 자기가 맡은 기사 하나만 알면 됩니다.

예를 들어 기사 3을 담당하는 워커는:

기사 3 본문

만 알고,

기사 1은 무엇인지
기사 2는 무엇인지
기사 4는 무엇인지

를 알 필요가 없습니다.

17. 이것이 오히려 장점입니다

각 워커가 서로 독립적이기 때문에:

테스트하기 쉽습니다

기사 하나 → 취재 함수

만 따로 테스트할 수 있습니다.

서로 간섭하지 않습니다

기사 A의 요약이 기사 B의 요약에 영향을 주지 않습니다.

병렬 처리하기 좋습니다

각 기사를 동시에 처리할 수 있습니다.

18. 그런데 병렬화하면 비용이 줄어들까요?

아닙니다.

이것은 반드시 구분해야 합니다.

기사 5개를:

순차 처리

기사1 → 기사2 → 기사3 → 기사4 → 기사5

하는 대신:

```
동시 처리
기사1 ↙
기사2 ↳ 동시에
기사3 ↘
기사4 ↘
기사5 ↘
```

하면 시간은 줄어들 수 있습니다.

하지만 LLM 호출이 5번인 것은 같습니다.

즉,

▮ 병렬화는 주로 시간을 줄이는 것이지 토큰 비용을 없애는 것이 아닙니다.

19. ⚠ Fan-out의 경계를 어디에 둘 것인가?

이것이 이번 챕터에서 매우 중요한 설계 포인트입니다.

선별 단계

다른 기사들을 함께 봐야 합니다.

```
기사 A
기사 B
기사 C
기사 D
```

를 비교해야:

▮ "이 중 어떤 것이 더 중요한가?"

를 판단할 수 있습니다.

따라서 한 화면에 모아야 합니다.

취재/요약 단계

기사 하나만 봐도 됩니다.

기사 A → 본문 → 요약

따라서 펼쳐도 됩니다.

20. 팬아웃 경계의 결정 공식

아주 쉽게 기억하면 됩니다.

|"이 단계에서 다른 항목을 봐야 답할 수 있는가?"

YES

→ 모아서 처리

상대평가
중복 판단
TOP 5 선정

NO

→ 펼쳐서 처리

본문 추출
기사 요약
언어 검사

그래서 현재 구조는:

수집
↓
선별 ← 여러 후보를 함께 봐야 함
↓
★ Fan-out
↓
기사별 취재/요약
↓
검수

가 됩니다.

21. ! 잘못된 Fan-out의 위험

만약 선별 단계부터 기사를 하나씩 펼쳐버리면:

기사 A → 중요?
기사 B → 중요?
기사 C → 중요?

가 됩니다.

그러면 AI는 다른 기사를 볼 수 없습니다.

결과적으로:

| "오늘 기사 100개 중 TOP 5를 골라라."

가 아니라

| "이 기사 하나가 중요한가?"

라는 전혀 다른 문제가 됩니다.

즉, 프로그램은 정상적으로 실행되지만 업무 결과는 달라집니다.

이것은 챕터 05에서 배운 **실행 성공 vs 작업 성공**의 대표적인 사례입니다.

22. 본문이 부족한 기사는 어떻게 할까요?

취재 과정에서 어떤 기사는 본문을 제대로 가져오지 못할 수 있습니다.

예:

기사 1 → 본문 확보 ☒
기사 2 → 본문 확보 ☒
기사 3 → 본문 부족 ☒
기사 4 → 본문 확보 ☒
기사 5 → 본문 확보 ☒

이때 기사 3 때문에 전체 파이프라인을 멈추면 안 됩니다.

기사 3 → 빈 리스트

처럼 처리하고 나머지는 계속 진행할 수 있습니다.

그러면:

최종 발행 5개가 아니라
4개

가 될 수도 있습니다.

중요한 것은 **역지로 다른 기사를 다시 긁어오지 않는 것**입니다.

왜 빠졌는지를 로그에 남기고, 실제 품질을 유지하는 것이 더 중요합니다.

23. 이번 챕터에서 배우는 자동화 설계의 큰 원칙

이번 챕터를 한 단계 높여서 보면 다음과 같습니다.

① 구조를 먼저 설계한다

headline
summary
why

② 검증 가능한 형태로 만든다

구조화 출력

③ 독립적인 작업은 펼친다

Send

④ 서로 비교해야 하는 작업은 모아서 한다

상대평가

⑤ 실패한 작업은 전체 시스템과 분리한다

기사 1 실패
≠
전체 뉴스레터 실패

이것이 **안정적인 AI Agent Pipeline**의 기본 설계입니다.

24. 🤖 AI 개발자로서 왜 알아야 할까요?

이번 챕터는 단순히 "기사 요약하는 코드"를 배우는 것이 아닙니다.

앞으로 AI 서비스를 만들 때 계속 만나게 될 세 가지 문제를 배우는 것입니다.

문제 ① 출력이 제각각이다

→ **Structured Output**

문제 ② 데이터가 많다

→ **Sectioning / Fan-out**

문제 ③ 일부 작업이 실패한다

→ **부분 실패 격리 + 로그**

이 세 가지는 실제 AI Agent를 운영할 때 매우 자주 등장합니다.

25. 💡 사용자님의 MSM과 연결하면

MSM에서 사용자가 자료를 많이 넣는 상황을 생각해보겠습니다.

일기 100개
사진 200장
SNS 300개
음성 50개

모든 자료를 한꺼번에 처리할 필요는 없습니다.

예를 들어:

전체 자료
↓
핵심 사건 선별
↓

★ Fan-out

↓

자료 하나씩 분석

↓

문체/감정/인물/장소/사건 추출

↓

다시 통합

할 수 있습니다.

그리고 각 결과를:

원문
정리문
해석

처럼 구조화하면 나중에:

원문과 대조
문체 검사
사실관계 검사
웹소설 변환
웹툰 장면 생성

등을 자동화하기 쉬워집니다.

따라서 이번 챕터의 "출력 구조를 먼저 정한다"는 개념은 MSM 설계에서도 상당히 중요합니다.

핵심 정리

이번 챕터에서는 제목과 링크뿐인 뉴스를 실제 뉴스레터 콘텐츠로 변환하는 방법을 배웠습니다.

① RSS의 제목/링크만으로는 부족

→ 실제 본문을 가져와야 합니다.

② 요약 결과를 세 칸으로 분리

headline
summary

→ 각각 검증 방법이 다르기 때문입니다.

③ 출력 스키마는 검증 가능성을 결정

| 무엇을 어떤 칸에 넣느냐가 나중에 무엇을 자동 검사할 수 있는지를 결정합니다.

④ 구조화 출력

→ AI의 출력 형식을 안정적으로 만듭니다.

하지만 내용의 정확성까지 보장하지는 않습니다.

⑤ Send 로 Fan-out

→ 선별된 기사 수만큼 취재 작업을 펼칩니다.

⑥ Fan-out 경계

| 다른 항목을 함께 봐야 하면 모아서 처리하고, 하나만 봐도 되면 펼쳐서 처리합니다.

★ 이것만은 꼭 기억하세요

1. 제목과 링크만으로 AI가 요약하게 하면 환각 위험이 커집니다.
2. 요약 전에 원문 본문을 확보하는 것이 중요합니다.
3. **headline**, **summary**, **why** 처럼 검증 방법이 다른 결과는 서로 다른 필드로 분리합니다.
4. 출력 스키마는 단순한 형식 지정이 아니라 '무엇을 자동 검증할 수 있는가'를 결정합니다.
5. Structured Output은 형식을 보장하지만 내용의 진실성을 보장하지 않습니다.
6. 프롬프트는 요청이고, 코드 검사는 실제 규칙을 확인하는 장치입니다.
7. **Send** 는 기사 수에 따라 작업을 동적으로 펼치는 Fan-out 기능입니다.
8. Fan-out하면 처리 시간은 줄일 수 있지만 LLM 호출 비용 자체가 사라지는 것은 아닙니다.
9. 상대평가처럼 여러 기사를 함께 봐야 하는 단계는 펼치면 안 됩니다.
10. 본문 추출·요약처럼 기사 하나만 보면 되는 작업은 Fan-out하는 것이 효율적입니다.

한 문장으로 정리

뉴스레터 에이전트는 제목과 링크에서 멈추지 않고 본문을 확보한 뒤 검증 가능한 구조로 요약하며, 서로 독립적인 기사별 취재·요약 작업은 **Send** 를 이용해 Fan-out으로 펼쳐 처리해야 합니다.

Chapter09

이번 챕터는 “AI가 만든 요약이 정말 원문에 근거하고 있는가?”를 자동으로 검사하는 방법을 다룹니다. 특히 숫자 비교의 한계 → LLM 검수 → 검수 실패 처리 → 검수 자체를 검증하는 방법까지 연결해서 이해하시면 좋습니다.

/강의정리 Chapter 09

요약이 원문을 벗어났는지 검사하기

1. 이번 챕터에서 해결하려는 문제

AI가 만든 요약이 매끄럽다고 해서 정확한 것은 아닙니다.

예를 들어 AI가 원문에 없는 내용을 자연스럽게 만들어도 사람이 읽으면 그럴듯하게 느껴질 수 있습니다.

따라서 자동 뉴스레터 시스템에서는

| **생성 → 검수 → 통과한 결과만 발행**

이라는 구조가 필요합니다.

핵심은 **생성하는 AI와 검수하는 AI의 역할을 분리하는 것**입니다.

2. 가장 단순한 검수: 숫자 대조

처음 생각할 수 있는 방법은 아주 간단합니다.

| “요약에 나온 숫자가 원문에도 있는가?”

예를 들어 원문에

| `just three months after ... raised $60 million`

이 있다고 해보겠습니다.

AI가 한국어로

“지난 3개월 전 6000만 달러 규모의 투자 이후”

라고 요약했습니다.

문자열만 비교하면 문제가 발생합니다.

원문	요약
three months	3개월
\$60 million	6000만 달러

문자열은 다르지만 의미와 값은 같습니다.

따라서 숫자 대조 결과는

3, 6000 → 원문에 없음 → 환각 의심

이라고 판단합니다.

하지만 실제로는 정확한 번역과 단위 변환입니다.

핵심 교훈

검사가 틀린 것이 아니라 대조 방식이 문제입니다.

문자열 비교는

"6000" ≠ "60 million"

이지만,

의미상으로는

6,000만 = 60 million

입니다.

따라서 영어 원문을 한국어로 요약하는 시스템에서는 단순 문자열 비교만으로는 부족합니다.

3. 검수 방법 4가지

① 검수하지 않는다

가장 저렴하고 간단합니다.

하지만 뉴스레터처럼 신뢰가 중요한 서비스에서는 위험합니다.

한 번의 잘못된 정보가 구독자 이탈로 이어질 수 있습니다.

② 규칙 기반 검사

숫자나 고유명사를 원문과 비교합니다.

장점

- 비용 거의 없음
- 빠름
- 결과가 결정론적

단점

- 번역·단위 변환에 약함
- 숫자가 없는 거짓 정보는 발견하지 못함

예를 들어

└ “업계 최초의 기술이다.”

라는 문장은 숫자가 없기 때문에 숫자 검사를 통과합니다.

③ LLM 검수 ← 이번에 선택

원문과 요약을 LLM에게 함께 제공하고 묻습니다.

└ “요약의 각 주장이 원문에서 뒷받침되는가?”

이 방식은 문자열이 아니라 의미를 비교합니다.

따라서

- three months → 3개월
- \$60 million → 6000만 달러

같은 정상적인 번역·변환은 통과시킬 수 있습니다.

또한

└ “업계 최초”

같은 숫자 없는 과장도 판단할 수 있습니다.

단점도 있습니다.

- LLM 호출 비용 발생

- 처리 시간이 증가
- 검수 LLM 자체도 틀릴 수 있음

즉,

| **검수 = 정확성 보장**

이 아니라

| **검수 = 잘못된 결과가 나갈 확률을 낮추는 장치**

입니다.

④ 인용 강제

요약을 만들 때 각 문장마다

| “이 문장을 뒷받침하는 원문의 근거 문장을 함께 제시하라.”

고 요구합니다.

그리고 실제 원문에 그 근거가 있는지 문자열로 확인합니다.

장점은 근거를 직접 확인할 수 있다는 것입니다.

하지만 출력이 길어지고 비용이 증가하며, AI가 근거 문장 자체를 만들어낼 가능성도 있습니다.

또한 요약이 원문 표현에 너무 묶일 수 있습니다.

4. 왜 ③ LLM 검수를 선택했을까?

이번 뉴스레터는 **영어 원문 + 한국어 요약** 구조입니다.

따라서 단순 문자열 비교인 ②는 구조적으로 오탐이 많습니다.

④는 정확하지만 요약의 자유도를 떨어뜨립니다.

①은 뉴스레터의 신뢰성 측면에서 위험합니다.

그래서

| **원문과 요약을 함께 넣고 LLM이 의미적으로 검수**

하는 ③을 선택했습니다.

5. 그런데 **why** 는 왜 검수하지 않을까?

섹션 8에서 결과를

- **headline**
- **summary**
- **why**

로 나누었습니다.

이번 검수에서는

| **headline + summary**만 검사

합니다.

왜냐하면 **why** 는 독자를 위한 해석과 설명이기 때문입니다.

예를 들어 AI가

| “이 기술은 앞으로 콘텐츠 산업을 크게 변화시킬 가능성이 있습니다.”

라고 설명했다고 해보겠습니다.

이 문장은 원문에 그대로 존재하지 않을 수 있습니다.

따라서 이것을 원문과 똑같이 검수하려 하면 계속 불합격할 가능성이 높습니다.

즉,

| **사실 주장** → 검수 대상

| **해석·의견** → 별도의 기준 필요

라는 구분입니다.

6. 검수는 생성과 분리해야 한다

중요한 원칙입니다.

생성 AI에게

| “정확하게 작성하세요.”

라고 부탁하는 것만으로는 부족합니다.

프롬프트는 **요청이지 보장**이 아니기 때문입니다.

또한 AI에게

“네가 만든 요약이 정확해?”

라고 물으면 자신이 만든 결과를 정당화할 가능성이 있습니다.

그래서 구조를 이렇게 만듭니다.

```
원문
↓
취재/요약
↓
요약 결과
↓
별도의 검수
↓
합격 → 발행
불합격 → 버림
```

즉,

생성자와 평가자를 분리

합니다.

이것이 앞에서 배운 **평가자-최적화자 패턴**과 연결됩니다.

7. 검수 결과도 구조화한다

검수 결과를 그냥

“괜찮습니다.”

라고 받으면 자동화하기 어렵습니다.

따라서 다음처럼 구조화합니다.

```
{
  ok: true/false,
  problems: [...]
}
```

예를 들어:

```
{
  "ok": false,
  "problems": [
    "투자 금액이 원문과 다릅니다."
  ]
}
```

이렇게 하면 시스템이 자동으로

`ok = true` → 발행

`ok = false` → 제외

할 수 있습니다.

중요한 원칙

AI에게 구조화된 출력을 요구하면 후속 자동화가 쉬워집니다.

8. 불합격한 요약을 어떻게 처리할까?

세 가지 방법이 있습니다.

① 다시 작성

검수에서 발견한 문제를 알려주고 다시 요약합니다.

장점은 결과를 살릴 수 있다는 것.

단점은 비용과 복잡도가 증가합니다.

② 취재 노드로 되돌리기

그래프에 다시 돌아가는 경로를 만듭니다.

하지만 한 건 때문에 전체 프로세스가 다시 돌 수 있습니다.

③ 그냥 버리고 로그에 기록 ← 선택

이번 시스템에서는 이것을 선택했습니다.

이유는 단순함입니다.

다섯 건을 반드시 채우기 위해 계속 재시도하지 않습니다.

예를 들어

```
5개 취재
↓
4개 요약 성공
↓
4개 검수 통과
↓
4개만 발행
```

이면 그냥 **그날은 4건 발행**합니다.

이것이 중요한 설계 철학입니다.

▮ 완벽하게 5건을 채우는 것보다, 검증된 4건만 보내는 것이 낫다.

9. **drafted** 와 **verified** 를 분리한 이유

여기서 LangGraph의 State 설계와 연결됩니다.

drafted에는 여러 취재 워커의 결과를 모으기 위해 **리듀서**가 붙어 있습니다.

즉,

```
drafted + 결과
drafted + 결과
drafted + 결과
```

처럼 계속 합쳐집니다.

그런데 검수는 반대입니다.

```
5개 drafted
↓ 검수
3개만 통과
```

즉 **줄이는 작업**입니다.

따라서 같은 키를 사용하면 문제가 생깁니다.

그래서

```
drafted
↓
검수
```

↓
verified

로 분리했습니다.

기억할 것

| 모으는 데이터와 걸러내는 데이터는 별도의 State로 관리할 수 있다.

그리고 앞에서 배운 원칙대로 각 노드는 자기가 바꾼 State 키만 반환하면 됩니다.

10. 검수 시스템도 검수해야 한다

이 부분이 이번 챕터에서 특히 중요합니다.

검수 장치를 만들었다고 해서 그것이 제대로 작동한다고 생각하면 안 됩니다.

대표적인 지표가 있습니다.

① **verify_fail**

검수에서 떨어진 건수입니다.

- 항상 0 → 검수가 제대로 작동하지 않을 수도 있음
- 지나치게 높음 → 검수 기준이 너무 엄격하거나 앞 단계가 잘못됨

따라서

| **불합격 0건 = 무조건 좋은 결과**

가 아닙니다.

② **재작성 후 합격률**

불합격한 결과를 다시 작성했을 때 얼마나 통과하는지 확인합니다.

계속 불합격한다면 문제의 원인이 요약이 아니라 **본문 추출 단계**일 수도 있습니다.

③ **일부러 가짜 오류를 넣어보기**

가장 중요한 테스트입니다.

예를 들어 원래

| "기업 가치는 약 5억 달러"

인데 일부러

| “기업 가치는 약 50억 달러”

로 바꿉니다.

그리고 검수 시스템에 넣습니다.

그런데도

| `ok: true`

가 나온다면 검수 장치가 제대로 작동하지 않는 것입니다.

이것을 **의도적으로 오류를 주입하는 테스트**라고 생각하면 쉽습니다.

11. 왜 이런 테스트가 중요한가?

검수 시스템은 평소에 아무 문제가 없으면

| “정상적으로 작동하고 있겠지.”

라고 생각하기 쉽습니다.

하지만 실제로는 고장 나 있어도 아무 표시가 안 날 수 있습니다.

이것이 앞에서 배운

| **조용한 실패(Silent Failure)**

와 연결됩니다.

그래서 좋은 자동화 시스템은

| **정상적인 데이터만 테스트하지 않고 일부러 잘못된 데이터도 넣어본다.**

12. 객관식 정답

정답은 ②번입니다.

| 검사는 정상 동작했지만 문자열 일치라는 대조 방식이 번역·단위 변환을 다룰 수 없는 것이므로, 의미 수준의 대조로 바뀌어야 한다.

핵심은 **검사 코드의 오류와 검사 방법의 한계**를 구분하는 것입니다.

13. 주관식 질문의 핵심 답변

why 를 검사하지 않는 것은 분명 위험할 수 있습니다.

왜냐하면 AI가 독자에게 영향을 주는 해석을 잘못 만들 수 있기 때문입니다.

다만 **why** 를 원문의 사실 여부와 동일한 방식으로 검수하면 문제가 생깁니다.

따라서 별도의 안전장치를 두는 것이 좋습니다.

예를 들어:

headline / summary

→ 사실성 검수

why

→ "원문에 없는 새로운 사실을 추가하지 않았는가?"

"과도한 단정이나 과장이 없는가?"

"의견과 사실을 구분했는가?"

처럼 **검수 기준을 다르게 적용**할 수 있습니다.

이것은 앞으로 AI 서비스를 설계할 때 매우 중요한 사고방식입니다.

모든 출력물을 똑같은 기준으로 검사할 필요는 없다.데이터의 성격에 따라 검수 기준을 설계해야 한다.

14. 이번 챕터의 전체 구조

전체 뉴스레터 에이전트를 보면 점점 이런 구조가 됩니다.

RSS

↓

기사 수집

↓

본문 추출

↓

취재/요약

↓

검수

↓

통과한 기사만

↓

뉴스레터 발행

여기서 핵심적인 변화는

- *"AI가 만들었다"에서 끝나지 않고

"AI가 만든 것을 다시 평가한다"***는 것입니다.

★ 이번 챕터에서 꼭 기억할 7가지

1. 매끄러운 문장 ≠ 정확한 문장
2. 문자열 비교는 번역·단위 변환 때문에 오탐할 수 있습니다.
3. 영어+한국어 환경에서는 의미 기반 LLM 검수가 유리합니다.
4. 생성과 검수는 분리해야 합니다.
5. 검수 결과는 `{ok, problems}` 처럼 구조화하면 자동화하기 쉽습니다.
6. 불합격 결과를 반드시 재작성할 필요는 없습니다. 버리는 것도 좋은 설계입니다.
7. 검수 시스템도 일부러 틀린 데이터를 넣어 스스로 테스트해야 합니다.

🎯 AI 개발자 관점에서 가장 중요한 한 문장

AI 에이전트의 품질은 "얼마나 잘 생성하느냐"뿐 아니라 "잘못 생성된 결과를 얼마나 잘 발견하고 차단하느냐"까지 포함합니다.

한 줄 요약

이번 챕터는 AI가 만든 요약물을 그대로 믿지 않고, 별도의 검수 노드를 통해 원문과 의미적으로 대조한 뒤 검증된 결과만 발행하는 구조를 배운 것입니다.

Chapter10

이번 챕터는 뉴스레터 에이전트의 마지막 단계인 '발행(Publish)'을 완성하면서, 실제 외부 시스템과 연결할 때 필요한 웹훅·보안·한도·Dry Run·실패 처리를 배우는 내용입니다.

/강의정리 Chapter 10

마지막 노드 — 발행

1. 이번 챕터의 핵심

앞에서는 데이터를 수집하고 → 본문을 가져오고 → 요약하고 → 검수했습니다.

이제 마지막으로

검수된 결과를 실제 사용자에게 전달

합니다.

여기서 중요한 차이가 있습니다.

앞 단계가 잘못 실행되어도 대부분 프로그램 내부에서 끝나지만,

발행은 실제 외부 세계에 영향을 줍니다.

한 번 보내면 실제 사람의 화면에 나타나기 때문입니다.

따라서 발행 노드는 다른 노드보다 더 조심스럽게 설계해야 합니다.

2. 발행 건수는 단순하게

이번 시스템에서는 검수를 통과한 만큼 그대로 발행합니다.

예를 들어:

```
5건 수집
↓
4건 본문 추출
↓
4건 요약
↓
3건 검수 통과
↓
3건 발행
```

억지로 5건을 채우려고 다시 취재하지 않습니다.

그리고 **0건인 날**도 중요합니다.

아무것도 보내지 않으면

“오늘 뉴스가 없는 것이지”

“파이프라인이 고장 난 것이지”

구분할 수 없습니다.

그래서 0건이면

“오늘은 조용합니다.”

라는 메시지 하나를 발행합니다.

핵심 원칙

| 결과가 없다는 것도 하나의 결과로 알려준다.

3. 어디로 발행할 것인가?

네 가지 선택지가 있습니다.

방법	장점	단점
이메일	진짜 뉴스레터에 가장 적합	초기 설정이 복잡
Slack	팀 업무에 편리	관리자 승인 등이 필요할 수 있음
Discord 웹훅	가장 빠르게 구현 가능	뉴스레터 서비스에는 부족
정적 페이지	기록과 공유에 좋음	알림이 없음

이번에는 **Discord 웹훅**을 선택했습니다.

중요한 것은

| **Discord가 최종적으로 가장 좋은 발행 수단이라서가 아닙니다.**

오늘 안에 실제 결과를 받아 보는 것이 목적이기 때문입니다.

나중에 이메일로 바꾸더라도

수집
↓
요약
↓
검수
↓
publish
↓
Discord

에서

수집
↓
요약
↓
검수

```
↓  
publish  
↓  
Email
```

처럼 **publish** 노드의 외부 출력 부분만 교체할 수 있습니다.

이것이 좋은 설계입니다.

4. 웹훅이란?

웹훅은 쉽게 말하면

“이 주소로 메시지를 보내면 상대 시스템에서 정해진 일이 일어나게 하는 주소”

입니다.

예를 들어 Discord가 웹훅 주소를 하나 줍니다.

```
https://discord.com/api/webhooks/...
```

그리고 프로그램이 이 주소로 JSON을 **POST** 합니다.

그러면 Discord 채널에 메시지가 나타납니다.

즉,

```
내 프로그램  
|  
| HTTP POST + JSON  
↓  
Discord Webhook URL  
|  
↓  
Discord 채널
```

입니다.

왜 웹훅이 편리할까요?

일반 API 연동보다 훨씬 단순합니다.

- 별도의 복잡한 SDK가 필요하지 않음
- URL을 하나 받아두면 됨

- JSON 형식만 맞추면 됨
- `requests` 같은 HTTP 도구로 전송 가능

따라서 **알림·모니터링·배포 통지**처럼 한 방향으로 정보를 보내는 자동화에 특히 유용합니다.

5. 웹훅 URL은 비밀번호처럼 관리

이 부분은 반드시 기억해야 합니다.

웹훅 URL 자체가 사실상 **인증 수단**입니다.

따라서

```
DISCORD_WEBHOOK_URL = "https://discord.com/api/webhooks/..."
```

를 코드에 그대로 넣고 GitHub에 올리면 안 됩니다.

누군가 URL을 가져가면 해당 채널에 메시지를 보낼 수 있습니다.

따라서

| 웹훅 URL = 열쇠

라고 생각하면 쉽습니다.

유출됐다면 해당 웹훅을 삭제하고 새 웹훅을 만들어야 합니다.

앞에서 배운 API Key와 환경변수 관리와 같은 원칙입니다.

6. 첫 번째 테스트는 한 줄부터

처음부터 완성된 뉴스레터를 보내려고 하지 않습니다.

먼저

| “외부 시스템에 메시지를 보낼 수 있는가?”

만 확인합니다.

성공하면 Discord에서 메시지를 확인할 수 있습니다.

여기서 재미있는 것이 HTTP 응답 코드입니다.

204

처음 보면

“204? 실패한 것 아닌가?”

라고 생각하기 쉽습니다.

하지만 아닙니다.

204 No Content = 요청은 성공했지만 돌려줄 본문이 없음

이라는 뜻입니다.

즉,

```
204
↓
전송 성공
↓
응답 본문은 없음
↓
Discord 채널 확인
```

입니다.

7. HTTP 상태 코드로 실패 원인 찾기

발행 자동화에서는 오류를 숫자로 빠르게 분류할 수 있습니다.

400

보낸 JSON의 형식 등이 잘못되었을 가능성이 있습니다.

401 / 404

웹훅 주소가 잘못되었거나 웹훅이 삭제된 경우 등을 의심합니다.

429

요청을 너무 많이 보내서 **Rate Limit**에 걸린 경우입니다.

즉,

“조금 기다렸다가 다시 보내세요.”

라는 의미입니다.

중요한 생각

외부 API를 연결할 때는 단순히

| 성공 / 실패

만 보는 것이 아니라

| 실패했을 때 어떤 상태 코드가 나오는가

까지 설계해야 합니다.

8. **content** 보다 **embed** 가 좋은 이유

문자열 하나로 다섯 개 기사를 붙이면 이런 느낌입니다.

```
기사1...
기사2...
기사3...
기사4...
기사5...
```

읽기 어렵습니다.

Discord의 **Embed**를 사용하면 카드처럼 구성할 수 있습니다.

이번 구조에서는 섹션 9에서 만든 데이터 구조가 그대로 활용됩니다.

```
headline → 카드 제목
summary  → 카드 본문
why      → 강조 문구
url      → 기사 링크
footer   → 출처/시간
```

즉,

| 앞에서 데이터를 잘 구조화해 둔 것이 마지막 발행 단계에서 가치가 생깁니다.

이것이 이번 챕터에서 중요한 설계 연결입니다.

9. 발행에는 한도가 있다

외부 시스템에 데이터를 보낼 때는 반드시 **API 제한**을 확인해야 합니다.

Discord Embed에도

- Embed 개수 제한
- 제목 길이 제한
- 본문 길이 제한
- 전체 메시지 길이 제한

등이 있습니다.

따라서

| 만들어서 바로 보내는 것이 아니라, 보내기 전에 한도를 확인해야 합니다.

예를 들어 기사가 너무 많거나 텍스트가 지나치게 길면 **400** 오류가 발생할 수 있습니다.

일반화하면

AI 자동화에서 외부 시스템으로 보내는 마지막 단계에는 항상

```

데이터 생성
↓
데이터 정리
↓
한도 확인
↓
외부 전송
  
```

이라는 방어 단계가 필요합니다.

10. Dry Run은 왜 중요한가?

이번 코드의 중요한 기능이

| `dry_run`

입니다.

쉽게 말하면

| “실제로 보내지는 말고, 무엇을 보낼지만 보여줘.”

입니다.

예를 들어

[dry-run] embed 3개 · 590자 – 보내지 않음

이라고 나오면 실제 Discord에는 아무것도 보내지 않습니다.

이 기능은 자동화 시스템에서 매우 중요합니다.

프롬프트를 수정할 때마다 실제 채널에

```
테스트 기사  
테스트 기사  
테스트 기사  
...
```

가 계속 쌓이면 곤란합니다.

따라서

```
개발/테스트  
→ dry_run=True  
  
실제 발행  
→ dry_run=False
```

로 분리합니다.

11. 더 중요한 안전장치: 기본값을 “보내지 않음”으로

이번 코드에서는

```
DRY_RUN = "1"
```

을 기본값으로 두었습니다.

즉,

기본적으로는 발행하지 않습니다.

실제로 발행하려면 명시적으로

```
DRY_RUN = "0"
```

으로 바꿉니다.

이것은 자동화 시스템에서 매우 좋은 원칙입니다.

❗ 실수로 발행하는 것보다 실수로 발행하지 않는 것이 낫다.

특히 나중에 매일 새벽 자동 실행을 붙이면 이 차이가 매우 중요해집니다.

12. **publish** 노드는 필요한 데이터만 가져온다

State에는 여러 정보가 있을 수 있습니다.

예를 들어

```
verified  
body  
원문  
중간 처리 정보  
로그  
...
```

가 있을 수 있습니다.

그런데 Discord에 보낼 것은

```
headline  
summary  
why  
url  
source
```

정도입니다.

따라서 **publish**가 State 전체를 그대로 보내면 안 됩니다.

필요한 값만 골라 새로운 Embed를 만듭니다.

이것이 중요한 이유

1. 불필요한 정보가 노출되지 않음
2. 메시지가 길어지는 것을 방지

3. 외부 시스템의 데이터 한도 관리
4. 어떤 데이터를 외부로 내보내는지 명확해짐

즉,

▮ 내부 State와 외부로 나가는 데이터는 분리하는 것이 좋습니다.

13. 전체 파이프라인이 완성됐다

이제 전체 구조가 완성됩니다.

```
RSS
↓
수집
↓
본문 추출
↓
취재/요약
↓
검수
↓
verified
↓
publish
↓
Discord
```

각 단계의 역할도 명확합니다.

단계	역할
수집	어떤 기사가 있는가?
본문 추출	실제 내용을 가져왔는가?
취재	무엇을 전달할 것인가?
검수	원문에 근거하는가?
발행	사용자에게 어떻게 전달할 것인가?

이렇게 보면 하나의 AI 에이전트가 모든 것을 하는 것이 아니라,

▮ 각 노드가 하나의 책임을 가지고 연결된 파이프라인

이라는 것이 보입니다.

14. 이번 챕터에서 가장 중요한 설계 원칙

① 외부 세계에 영향을 주는 노드는 더 조심한다

발행은 단순한 데이터 처리와 다릅니다.

| External Side Effect

가 발생하기 때문입니다.

② Dry Run을 만든다

실제 실행과 테스트 실행을 분리합니다.

③ 기본값은 안전한 방향으로

기본값 → 보내지 않음
명시적으로 설정 → 실제 발행

이런 설계를 **Fail Safe**하게 만든다고 생각하면 됩니다.

④ 외부 API의 한도를 코드에 반영한다

API를 호출하는 것만으로 끝나는 것이 아니라

| “얼마까지 보낼 수 있는가?”

를 알아야 합니다.

⑤ 내부 State와 외부 출력은 분리한다

State 전체를 외부에 던지는 것이 아니라

| 발행에 필요한 데이터만 골라서 새로운 출력으로 만든다.

15. 객관식 정답

정답은 ②번입니다.

| 204는 성공했고 돌려줄 내용이 없다는 뜻이므로 정상이며, 채널을 확인하면 메시지가 올라와 있습니다.

204 No Content 는 실패 코드가 아닙니다.

더 해보기에서 꼭 해볼 것

두 실습 중에서는 2번을 특히 추천합니다.

일부러 20건을 넣어보기

왜냐하면 이것은 단순한 기능 실습이 아니라

“내 시스템이 한계를 만났을 때 어떻게 행동하는가?”

를 확인하는 테스트이기 때문입니다.

AI 자동화 시스템은 정상적인 데이터만 처리해서는 충분하지 않습니다.

정상 데이터
+
너무 많은 데이터
+
빈 데이터
+
잘못된 데이터
+
외부 API 오류

를 모두 생각해야 합니다.

★ 이번 챕터에서 꼭 기억할 8가지

1. 발행은 외부 세계에 영향을 주는 노드이므로 특별히 조심해야 합니다.
2. 처음에는 **Discord 웹훅**처럼 가장 단순한 방식으로 실제 발행 경험을 만드는 것이 좋습니다.
3. 웹훅 URL 자체가 비밀번호와 같은 역할을 하므로 절대 공개하면 안 됩니다.
4. 204 는 Discord 발행 성공을 의미할 수 있으며 실패가 아닙니다.
5. embed 를 이용하면 headline / summary / why / source 를 구조적으로 표현할 수 있습니다.
6. Dry Run은 자동화 시스템에서 중요한 안전장치입니다.
7. 외부 시스템의 API 한도와 오류 코드까지 설계에 포함해야 합니다.

8. 내부 State와 외부 발행 데이터를 분리하면 시스템이 안전하고 명확해집니다.

AI 개발자 관점에서 가장 중요한 것

이번 챕터를 통해 에이전트 개발의 관점이 한 단계 넓어집니다.

단순히

“AI가 결과를 만들었다.”

에서 끝나는 것이 아니라,

“AI가 만든 결과를 안전하게 외부 시스템으로 전달하는 마지막 단계까지 설계했다.”

가 됩니다.

특히 앞으로 선생님께서 **1인 AI 컨설팅**이나 **업무자동화 서비스**를 만들 때 이 개념이 중요합니다.

예를 들어 고객의 업무를 자동화한다면

```
AI 분석
↓
AI 결과 생성
↓
검수
↓
승인/안전장치
↓
이메일 발송
↓
CRM 등록
↓
고객에게 알림
```

처럼 **AI의 판단**과 **실제 행동**을 분리해야 합니다.

한 문장으로 정리하면

좋은 AI 에이전트는 똑똑하게 생성하는 것에서 끝나는 것이 아니라, 검증된 결과를 안전하게 외부 세계로 내보내는 것까지 설계된 시스템입니다.

Chapter11

이번 챕터는 완성된 뉴스레터 에이전트를 사람이 실행하지 않아도 매일 자동으로 돌게 만드는 방법입니다. 핵심은 단순히 스케줄을 거는 것이 아니라 **자동 실행 + 비밀값 관리 + 실행 여부 확인(관측)**을 함께 설계하는 것입니다.

/강의정리 Chapter 11

자동 실행에 필요한 것

1. 이번 챕터의 핵심

지금까지 만든 파이프라인은 사람이 노트북을 열고 실행해야 했습니다.

이제 이것을

■ **매일 아침 7시 30분 → 자동 실행 → 뉴스 수집 → 요약 → 검수 → Discord 발행**

하도록 만듭니다.

이를 위해 필요한 것은 세 가지입니다.

- ① 언제 실행할 것인가?
- ② API 키를 어떻게 안전하게 전달할 것인가?
- ③ 제대로 실행됐는지 어떻게 확인할 것인가?

특히 ③ **관측(Observability)**이 중요합니다.

자동화는 사람이 보지 않기 때문에

■ **멈춰도 모를 수 있기 때문입니다.**

2. GitHub Actions를 선택한 이유

이번에는 **GitHub Actions**를 사용합니다.

GitHub 저장소에 코드를 올려두면 별도의 컴퓨터를 켜 놓지 않아도 GitHub의 실행 환경에서 코드를 돌릴 수 있습니다.

전체 구조는 다음과 같습니다.

```
GitHub Repository
  ↓
GitHub Actions
  ↓
```

```
정해진 시간에 실행
↓
Python 프로그램
↓
뉴스 수집 → 요약 → 검수
↓
Discord 발행
```

즉,

내 컴퓨터가 꺼져 있어도 GitHub가 대신 실행해 주는 구조

입니다.

3. 노트북을 Python 파일로 바꾼다

Jupyter Notebook은 사람이 셀을 순서대로 실행하는 것을 전제로 합니다.

자동 실행 환경에서는 적합하지 않습니다.

그래서 먼저

```
notebook.ipynb
↓
graph.py
```

로 옮깁니다.

구조는 대략 다음과 같습니다.

```
my-newsletter/
|
├─ graph.py
├─ run.py
├─ requirements.txt
├─ store/
|
├─ .github/
│   └─ workflows/
│       └─ daily.yml
```

각각의 역할

`graph.py`

실제 뉴스레터 파이프라인입니다.

- State
- 노드
- Graph
- 실행 함수

등이 들어갑니다.

`run.py`

실행 진입점입니다.

쉽게 말하면

“그래프를 실제로 한 번 실행해!”

라고 하는 파일입니다.

`requirements.txt`

필요한 Python 라이브러리를 적어 놓습니다.

예:

```
feedparser
requests
trafilatura
openai
langgraph
pydantic
```

4. Notebook을 옮길 때 주의할 것

Notebook에서는 같은 이름을 다시 정의하는 것이 가능합니다.

예를 들어 중간에

```
build()
```

를 정의했다가 나중에 다시 정의할 수 있습니다.

Notebook에서는 마지막 정의가 사용됩니다.

하지만 Python 파일에서는 이것을 정리해야 합니다.

최종 버전 하나만 남긴다.

또한 순서도 중요합니다.

```
클래스/상수 정의
↓
함수 정의
↓
Graph 구성
↓
실행
```

처럼 사용하기 전에 정의되어 있어야 합니다.

5. 실습용 코드와 실제 실행 코드를 구분한다

Notebook에는 실습을 위해 이런 코드가 있을 수 있습니다.

```
for title in titles:
    ...
```

이런 것은 자동 실행 프로그램에 필요하지 않습니다.

반면

```
Pick
Shortlist
Draft
SYS
```

같은 클래스와 상수는 실제 노드가 사용하기 때문에 가져가야 합니다.

즉,

실습용 반복문은 제거하고, 실제 시스템에 필요한 정의만 가져간다.

이것이 Notebook → 프로그램 전환의 중요한 과정입니다.

6. `run.py` 를 따로 만드는 이유

`graph.py` 안에서 바로

```
run()
```

을 실행하도록 만들면 위험합니다.

다른 파일에서

```
import graph
```

하는 순간 브리핑이 실행될 수 있기 때문입니다.

따라서

```
if __name__ == "__main__":
```

이라는 진입점을 둡니다.

쉽게 말하면:

“이 파일을 직접 실행했을 때만 실제 작업을 시작해라.”

입니다.

이 원칙은 Python 자동화에서 매우 중요합니다.

7. API Key는 코드에 넣지 않는다

이번 챕터에서 반드시 기억해야 하는 부분입니다.

다음처럼 작성하면 안 됩니다.

```
OPENAI_API_KEY = "sk-....."
```

또는

```
DISCORD_WEBHOOK_URL = "https://discord.com/api/webhook  
s/..."
```

GitHub 저장소가 공개되어 있다면 특히 위험합니다.

API 키는

GitHub Secrets

에 넣습니다.

```
GitHub
↓
Settings
↓
Secrets and variables
↓
Actions
↓
New repository secret
```

그리고

```
OPENAI_API_KEY
DISCORD_WEBHOOK_URL
```

두 값을 등록합니다.

8. Secrets의 핵심 개념

Secrets는 단순히

“코드에서 안 보이게 하는 기능”

정도로 이해하면 안 됩니다.

자동화 시스템에서

코드와 비밀정보를 분리하는 구조

입니다.

```
코드
├─ graph.py
├─ run.py
└─ daily.yml

비밀
├─ OPENAI_API_KEY
└─ DISCORD_WEBHOOK_URL
```

이렇게 분리합니다.

그리고 실행할 때만 환경변수로 전달합니다.

9. 이미 GitHub에 올린 키는 더 조심해야 한다

중요한 보안 원칙입니다.

API Key를 실수로 Git에 커밋했다면

“파일에서 지우면 됐겠지.”

라고 생각하면 안 됩니다.

Git은 **과거 커밋 이력**을 가지고 있기 때문입니다.

따라서 이미 공개된 키는

삭제하고 새 키를 발급받는 것

이 안전합니다.

즉,

비밀정보는 처음부터 저장소에 들어가지 않게 하는 것이 가장 좋습니다.

10. GitHub Actions의 **schedule**

자동 실행 시간은 Cron으로 정합니다.

이번 예제는 한국시간 오전 7시 30분입니다.

그런데 GitHub Actions의 Cron은 **UTC 기준**입니다.

한국은 UTC보다 9시간 빠르므로

KST 07:30

↓

UTC 22:30

이고 Cron은

30 22 * * *

가 됩니다.

여기서 날짜가 전날로 넘어가는 것도 기억해야 합니다.

즉,

| 한국시간 9월 15일 오전 7:30 = UTC 9월 14일 22:30

입니다.

11. `workflow_dispatch`가 중요한 이유

자동 실행만 만들어 놓으면 테스트하기 불편합니다.

예를 들어 코드를 수정했는데

| “내일 아침까지 기다려서 확인해 보자.”

하면 개발이 너무 느립니다.

그래서

```
schedule
+
workflow_dispatch
```

를 함께 넣습니다.

`workflow_dispatch`는 쉽게 말하면

| “사람이 버튼을 눌러 지금 실행할 수 있게 하는 기능”

입니다.

그래서 개발 과정은

```
수정
↓
Run workflow
↓
dry-run
↓
로그 확인
↓
문제 없으면 실제 발행
```

으로 빠르게 반복할 수 있습니다.

꼭 기억할 것

자동화의 첫 단계는 자동으로 돌리는 것이 아니라, 사람이 원할 때도 실행할 수 있게 만드는 것입니다.

12. `timeout-minutes` 가 필요한 이유

AI API나 웹사이트를 호출하다 보면 응답이 예상보다 오래 걸릴 수 있습니다.

그런데 종료 조건이 없다면 프로그램이 계속 기다릴 수 있습니다.

그래서

```
timeout-minutes: 20
```

처럼 제한을 둡니다.

즉,

20분 안에 끝나지 않으면 실행을 종료

합니다.

이것은 비용과 자원 낭비를 막는 안전장치입니다.

13. `if: always()` 가 중요한 이유

실행 기록을 파일로 남긴다고 해보겠습니다.

그런데 프로그램이 실패하면 마지막 단계가 실행되지 않을 수도 있습니다.

그러면 정작 우리가 가장 보고 싶은

실패한 실행의 기록

을 잃어버립니다.

그래서

```
if: always()
```

를 사용합니다.

뜻은

| 앞 단계가 성공했든 실패했든 이 단계는 실행하라.

입니다.

따라서

실행

↓

성공 → 기록 저장

실패 → 그래도 기록 저장

이 됩니다.

아주 중요한 원칙

| 실패했을 때의 로그가 성공했을 때의 로그보다 더 중요할 수 있습니다.

14. 자동화에서 "관측"이 필요한 이유

이번 챕터에서 가장 중요한 개념 중 하나입니다.

자동화는 사람의 일을 줄여줍니다.

그런데 역설적으로

| 사람이 보지 않기 때문에 고장 나도 모릅니다.

예를 들어 뉴스레터가

월요일 정상

화요일 정상

수요일 정상

목요일 실패

금요일 실패

토요일 실패

...

해도 아무도 확인하지 않으면 몇 주 동안 모를 수 있습니다.

따라서 자동화는

| **Automation + Observability**

로 생각해야 합니다.

15. 실행 여부를 확인하는 3중 안전장치

이번 시스템에서는 세 가지 방법을 사용합니다.

① GitHub 실패 알림

실패하면 GitHub가 알림을 보냅니다.

“시끄럽게 실패하는 오류”를 잡습니다.

② 0건이어도 Discord 메시지 발행

앞 챕터에서 만든

“오늘은 조용합니다.”

가 여기서 다시 중요해집니다.

뉴스 없음
→ “오늘은 조용합니다.” 수신

시스템 자체가 죽음
→ 아무 메시지도 안 올

따라서 알림이 오는 것 자체가 실행 증명이 됩니다.

이것은 매우 저렴하면서 강력한 모니터링 방법입니다.

③ Actions 실행 목록 확인

GitHub Actions에서

- 초록색 → 성공
- 빨간색 → 실패

등으로 실행 상태를 확인할 수 있습니다.

16. 자동화의 역설

이번 챕터의 핵심 철학을 하나로 정리하면 이것입니다.

자동화의 진짜 효과는 일을 보이지 않게 만드는 것입니다.

그런데

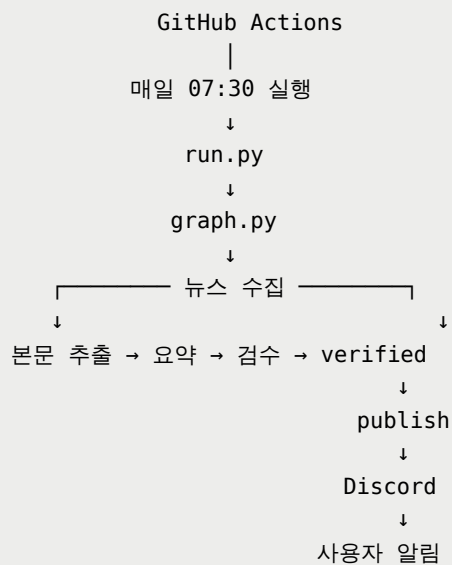
보이지 않는 일은 멈춰도 티가 나지 않습니다.

그래서 자동화할수록 관측 장치가 더 중요해집니다.

이것은 앞으로 선생님께서 업무자동화나 AI 에이전트를 만들 때도 그대로 적용됩니다.

17. 전체 구조

이제 시스템은 다음처럼 완성됩니다.



그리고 옆에서

```
GitHub Secrets
↓
API Key / Webhook
↓
안전하게 주입
```

하고,

```
Actions Log
+
store/
+
Discord 알림
```

으로 실행을 관찰합니다.

★ 이번 챕터에서 꼭 기억할 8가지

1. Notebook은 실험용, Python 파일은 자동 실행용으로 생각합니다.
2. `run.py` 라는 실행 진입점을 따로 두면 안전합니다.
3. API Key와 Webhook URL은 코드에 넣지 말고 GitHub Secrets에 저장합니다.
4. GitHub Actions의 Cron은 UTC 기준입니다.
5. `workflow_dispatch` 를 넣으면 기다리지 않고 수동 실행하여 테스트할 수 있습니다.
6. `timeout-minutes` 로 무한 대기과 불필요한 비용을 방지합니다.
7. `if: always()` 로 실패했을 때도 기록을 보존합니다.
8. 자동화는 반드시 관측과 함께 설계해야 합니다.

🎯 선생님께서 AI 개발자로서 특히 가져가실 개념

이번 챕터는 단순히 “GitHub Actions 사용법”을 배우는 수업이 아닙니다.

더 중요한 것은 다음 사고방식입니다.

```
AI 자동화
↓
자동 실행
↓
비밀정보 관리
↓
실패 방지
↓
실행 기록
↓
실행 여부 확인
```

즉,

“자동으로 실행되게 만드는 것”과 “자동으로 실행되고 있다는 것을 확인할 수 있게 만드는 것”은 별개의 문제입니다.

앞으로 선생님께서 ASP의 AI 컨설팅 자동화나 My-Story-Maker 같은 실제 AI 서비스를 만드실 때도 이 원칙이 그대로 적용됩니다.

한 문장 요약

좋은 자동화 시스템은 사람이 없어도 실행되는 시스템이 아니라, 사람이 없어도 실행되고 문제가 생기면 사람이 알아차릴 수 있는 시스템입니다.

Chapter12

/강의정리 Chapter 12 — 내린 결정을 다시 보기

이번 챕터의 핵심은 “한 번 정한 설계가 정말 좋은 결정이었는지, 실행 데이터를 쌓아서 다시 검증하자”입니다.

앞에서 RSS 사용, 관문 3개, 본문 600자, 후보 40건, 발행 5건 등을 결정했습니다. 하지만 대부분 **한두 번의 측정만으로 내린 결정**이었습니다.

좋은 AI 시스템은 “그때 그렇게 생각해서 정했다”가 아니라 “**실행 데이터를 보니 이 결정이 맞았다**”라고 설명할 수 있어야 합니다.

1. 실행할 때마다 데이터를 남긴다

가장 단순한 방법은 데이터베이스나 복잡한 모니터링 도구가 아니라

```
store/metrics.jsonl
```

파일에 **실행할 때마다 JSON 한 줄**을 추가하는 것입니다.

예를 들어 한 번의 실행 결과를 기록하면:

```
{
  "collected": 40,
  "picked": 12,
  "drafted": 8,
  "published": 5,
  "hours": 24,
  "by_source": {...},
  "log": [...]
}
```

처럼 남길 수 있습니다.

중요한 것은 숫자 하나가 아니라 **파이프라인의 흐름을 나중에 다시 볼 수 있게** 만드는 것입니다.

2. 가장 중요한 것: 깔때기(Funnel)

뉴스레터는 다음처럼 점점 후보가 줄어드는 구조입니다.

수집
↓
예선
↓
본문 취재/작성
↓
검수
↓
발행

숫자로 보면:

40건 수집
↓
12건 예선
↓
8건 초안
↓
5건 발행

이것이 바로 깔때기 관측입니다.

왜 중요한가?

예를 들어:

수집	40
예선	12
초안	8
발행	5

라면 각 단계에서 얼마나 탈락했는지 알 수 있습니다.

특히

- **picked → drafted** 차이가 크다 → 취재/본문 추출에서 문제가 있을 가능성
- **drafted → published** 차이가 크다 → 검수에서 탈락하는 기사가 많다는 의미

입니다.

즉,

“뉴스레터가 오늘 부실했다”가 아니라 “어느 단계에서 부실했는가?”를 알 수 있게 됩니다.

3. 숫자는 개수보다 비율이 중요하다

절대 숫자는 매일 뉴스의 양에 따라 달라집니다.

예를 들어:

월요일: 40 → 10 → 7 → 5
화요일: 80 → 20 → 14 → 5

수집량은 크게 다르지만,

예선 통과율
월요일: 25%
화요일: 25%

처럼 **비율은 비슷할 수 있습니다.**

그래서 다음과 같은 지표가 중요합니다.

예선 통과율 = picked / collected
초안 작성률 = drafted / picked
최종 발행률 = published / drafted

평소 예선 통과율이 25% 정도인데 어느 날 갑자기 60%가 됐다면,

후보가 달라졌거나 선별 기준이 느슨해졌을 가능성을 의심할 수 있습니다.

4. 깔때기는 “어디가 문제인지” 알려준다

하지만 한 가지 한계가 있습니다.

깔때기는 어디에서 문제가 발생했는지는 알려주지만, 왜 문제가 발생했는지까지 알려주지는 않습니다.

예를 들어:

```
수집 40
↓
예선 3
```

이라면 예선에서 많이 떨어졌다는 것은 알 수 있습니다.

하지만 이유는 모릅니다.

- 후보 뉴스 자체가 좋지 않았을까?
- 선별 기준이 너무 엄격했을까?
- RSS 소스가 바뀌었을까?

그래서 숫자와 함께 **탈락 이유를 로그로 남겨야 합니다.**

예:

```
dead source
본문 부족
본문 추출 실패
검수 실패
```

이런 로그가 있어야 숫자의 원인을 추적할 수 있습니다.

5. 소스별 성적표를 만든다

전체 깔때기와 별도로 “어떤 뉴스 소스가 실제로 도움이 되었는가?”도 봐야 합니다.

이미 저장해 둔 `by_source` 를 합산하면 됩니다.

예를 들어:

소스	발행 기여
Source A	28
Source B	15
Source C	3
Source D	0

이렇게 볼 수 있습니다.

특히 중요한 것은 **몇 주 동안 발행 기여가 0인 소스**입니다.

관문을 통과하고 본문도 충분한데 계속 0이라면,

기술적으로는 사용할 수 있지만 **우리 독자에게 가치 있는 기사를 거의 제공하지 않는 소스**일 가능성이 높습니다.

그러면 그 소스를 계속 유지할 이유가 있는지 다시 판단할 수 있습니다.

6. 한 소스에 지나치게 몰릴 수도 있다

반대로 이런 경우도 있습니다.

```
Source A → 70%
Source B → 15%
Source C → 10%
Source D → 5%
```

A가 거의 모든 발행 자리를 차지한다면 다른 소스는 후보 단계에서 사실상 의미가 없어질 수 있습니다.

이때는 다음과 같은 정책을 생각할 수 있습니다.

“한 소스에서 최대 2건까지만 발행한다.”

즉, 데이터를 보고 **파이프라인의 다음 설계 변경을 결정**하는 것입니다.

7. GitHub Actions에서는 기록이 사라진다

여기서 매우 중요한 현실적인 문제가 있습니다.

로컬 PC에서 실행하면:

```
내 컴퓨터
└─ store/metrics.jsonl
```

에 기록이 계속 쌓입니다.

하지만 GitHub Actions는 **매 실행마다 새로운 임시 서버**에서 실행됩니다.

따라서:

월요일 실행 → 임시 서버 → 기록 → 서버 삭제
화요일 실행 → 새로운 임시 서버 → 새로운 기록

이 됩니다.

즉, 실행 결과가 자동으로 누적되지 않습니다.

8. 그래서 metrics를 Git에 커밋한다

누적 기록을 남기려면 GitHub Actions가 실행 종료 후

```
store/metrics.jsonl
```

을 Git에 커밋하고 push하도록 만들 수 있습니다.

그러면:

```
월요일
metrics 기록
↓
Git commit

화요일
기존 기록 + 새로운 기록
↓
Git commit

수요일
기존 기록 + 새로운 기록
↓
Git commit
```

이 되어 시간이 지나면서 데이터가 누적됩니다.

그리고 `git pull` 을 하면 내 컴퓨터에도 기록이 따라옵니다.

9. 이것은 단순한 로그 이상의 의미가 있다

Git에 기록하면 단순히 숫자가 쌓이는 것뿐 아니라

“언제 어떤 변화가 있었는가”

까지 추적할 수 있습니다.

예를 들어:

```
9/14
예선 통과율 25%

9/20
예선 기준 변경

9/21
예선 통과율 40%
```

이런 식으로 보면

"기준을 바꾼 뒤 통과율이 어떻게 변했는가?"

를 되돌아볼 수 있습니다.

즉, **실험 기록 + 설계 의사결정 기록**이 되는 것입니다.

10. 단, 개인정보와 원문은 저장하지 않는다

Git에 저장한다는 것은 특히 주의해야 합니다.

`store/` 안에

- API 응답 원문
- 개인 정보
- 민감한 뉴스 데이터
- 불필요한 원문

등을 넣으면 그대로 저장소에 올라갈 수 있습니다.

따라서 Git에는 가능하면

```
metrics
통계
필요한 최소 로그
```

정도만 남기는 것이 좋습니다.

11. 이 챕터가 말하는 "결정 검증"의 핵심 지표 3가지

① 소스별 발행 기여

by_source

→ 어떤 소스가 실제로 결과물을 만들어내는가?

② 단계별 통과율

수집 → 예선 → 초안 → 발행

→ 어느 단계에서 많이 탈락하는가?

③ 발행 건수 분포

5건
5건
3건
0건
4건
...

→ 목표 발행량이 안정적으로 유지되는가?

특히 **0건인 날도 중요합니다.**

0건이라는 결과 자체가 문제일 수도 있지만,

| "오늘 뉴스가 없어서 0건인지, 시스템이 실행되지 않아서 0건인지"

구별할 수 있어야 합니다.

12. 이 챕터에서 가장 중요한 관점

앞에서는 이렇게 생각했습니다.

| "RSS가 좋을 것 같다."

“본문은 600자가 적당할 것 같다.”

“5건을 발행하면 될 것 같다.”

하지만 이제는 한 단계 더 나아갑니다.

가설
↓
설계
↓
실행
↓
측정
↓
데이터 축적
↓
검증
↓
설계 수정

이것이 **데이터 기반 AI 시스템 개선**입니다.

즉, AI 개발자는 처음부터 완벽한 구조를 만드는 사람이 아니라,

실행 결과를 측정하고, 그 결과를 근거로 시스템을 계속 개선하는 사람

이어야 합니다.

13. 사용자들이 AI 창업을 할 때 특히 중요한 이유

이 개념은 앞으로 만드실 **MSM(My-Story-Maker)**이나 AI 컨설팅 사업에도 그대로 연결됩니다.

예를 들어 MSM이라면:

입력한 글 100개
↓
분석 완료 90개
↓
스타일 분석 80개
↓
웹소설 변환 60개
↓
사용자 승인 45개

이런 식으로 볼 수 있습니다.

그러면

“MSM 사용자가 왜 중간에 이탈하는가?”

를 감으로 추측하는 것이 아니라 **데이터로 찾을 수 있습니다.**

AI 컨설팅에서도 마찬가지입니다.

문의 100건
↓
진단 70건
↓
제안 40건
↓
계약 15건
↓
실제 도입 10건

이렇게 보면 어느 단계가 병목인지 바로 드러납니다.

★ 꼭 기억해야 할 7가지

1. 설계 결정은 한 번의 실행만으로 확정하지 않는다.
2. 실행할 때마다 `metrics.jsonl`에 기록한다.
3. `collected` → `picked` → `drafted` → `published`는 **갈때기다**.
4. 절대 숫자뿐 아니라 **단계별 통과율**을 본다.
5. 숫자만으로 부족하므로 **탈락 이유**를 로그로 남긴다.
6. `by_source`를 이용하면 **소스별 성적표**를 만들 수 있다.
7. 자동화의 핵심은 실행뿐 아니라 **측정** → **검증** → **개선**까지 연결하는 것이다.

🎯 객관식 정답

정답: ②

기술적으로는 쓸 수 있는 소스지만 **우리 독자 기준에서 중요한 기사를 내놓지 않는다**는 뜻입니다.

관문 통과 + 본문 충분 + 발행 기여 0이 몇 주간 지속된다면, 기술적 장애보다는 **콘텐츠 가치 낮은 소스**일 가능성이 높습니다.

이번 챕터의 실습이 중요한 이유

`metrics.jsonl` 을 만드는 데 거창한 모니터링 시스템은 필요하지 않습니다.

핵심은 단순합니다.

```
with open(path, "a") as f:
    f.write(json.dumps(row, ensure_ascii=False) + "\n")
```

“측정할 수 있도록 기록하는 것”부터 시작하면 됩니다.

그리고 며칠 또는 몇 주 동안 데이터가 쌓이면 그때부터

정상 범위
↓
이상 징후
↓
원인 분석
↓
설계 변경

이라는 진짜 개선 사이클을 만들 수 있습니다.

한 문장으로 정리

좋은 AI 시스템은 한 번 잘 작동하는 시스템이 아니라, 실행 데이터를 계속 쌓아 자신의 설계가 맞는지 검증하고 더 나은 방향으로 수정할 수 있는 시스템입니다.

Chapter13

/강의정리 Chapter 13 — 구조와 내용 분리하기

이번 챕터의 핵심은 “프로그램의 구조와 분야별 내용을 분리하라”입니다.

지금까지 만든 뉴스레터에는 `사다리`, `관문`, `예선·본선`, `그래프`, `갈때기` 같은 범용적인 구조가 있습니다. 그런데 코드 안에는 “AI 뉴스”, “국내 개발팀”, “모델·API” 같은 특정 분야의 내용이 곳곳에 들어가 있습니다.

이 상태에서는 다른 분야로 바꾸려면 코드를 찾아다니며 수정해야 합니다.

1. 구조와 내용은 무엇이 다른가?

쉽게 비유하면 **식당의 주방과 메뉴**의 차이입니다.

- **구조** = 주방에서 음식을 만드는 방식
- **내용** = 오늘 판매하는 메뉴

주방의 조리 시스템은 그대로 두고 메뉴만 바꿀 수 있어야 합니다.

뉴스레터도 마찬가지입니다.

```
구조
수집 → 예선 → 취재 → 검수 → 발행
```

은 그대로 두고,

```
AI 뉴스
↓
채용 뉴스
↓
논문 뉴스
↓
콘텐츠 산업 뉴스
```

처럼 **내용만** 바꿀 수 있어야 합니다.

2. 설정을 어디에 둘 것인가?

이번 챕터에서는 4가지 방법을 비교했습니다.

① 코드 안에 직접 넣기

```
SYS = "당신은 국내 개발팀을 위한..."
```

가장 쉽습니다.

초기 프로토타입에서는 오히려 좋은 방법입니다.

하지만 내용이 여러 코드에 흩어지고, 내용을 바꾸려면 개발자가 코드를 수정해야 합니다.

② Python 상수로 모으기

```
AUDIENCE = "국내 개발팀"
```

```
CRITERIA = [...]
```

한 곳에 모을 수 있다는 장점이 있습니다.

하지만 여전히 **Python 코드**입니다.

기획자나 편집자가 수정하기에는 부담스럽습니다.

③ YAML/TOML 설정 파일로 분리 ★

이번에 선택한 방법입니다.

예:

```
audience.yaml
```

안에

```
독자
중요도 기준
버릴 것
토픽
데스크 지침
```

등을 적어둡니다.

그러면 프로그램은 이 파일을 읽어서 프롬프트를 만듭니다.

즉,

```
코드 = 어떻게 할 것인가
설정 = 무엇을 기준으로 할 것인가
```

로 나눌 수 있습니다.

④ 데이터베이스 + 관리자 화면

설정을 DB에 넣고 웹 화면에서 수정합니다.

여러 뉴스레터를 운영하거나 여러 사람이 설정을 관리할 때 적합합니다.

하지만 지금처럼 뉴스레터 하나를 운영하는 단계에서는 **너무 복잡합니다**.

3. 왜 YAML을 선택했는가?

가장 중요한 기준은

“누가 이것을 수정하는가?”

입니다.

예를 들어 독자와 중요도 기준은 개발자가 아니라 **편집 방향을 정하는 사람**이 수정해야 합니다.

그런데 Python 파일을 수정하게 하면:

```
기획자
↓
개발자에게 수정 요청
↓
개발자 코드 수정
↓
테스트
↓
배포
```

라는 불필요한 과정이 생깁니다.

YAML이라면:

```
기획자
↓
audience.yaml 수정
↓
다음 실행에 적용
```

할 수 있습니다.

4. 설정 파일에는 “값”뿐 아니라 “작성 규칙”도 적는다

이 부분이 이번 챕터에서 특히 중요합니다.

예를 들어 단순히

버릴 것:

- 홍보성 기사

라고 쓰는 것보다,

버릴 것:

- MOU·업무협약·투자유치·수상 등 당사자 홍보성 소식

이라고 쓰는 것이 훨씬 좋습니다.

왜냐하면 **“홍보성”이라는 말은 너무 추상적이기** 때문입니다.

AI 입장에서는 어떤 것이 홍보성인지 매번 다르게 판단할 수 있습니다.

반면

MOU / 업무협약 / 투자유치 / 수상

처럼 구체적으로 적으면 판단 기준이 훨씬 명확해집니다.

5. YAML 주석도 중요한 설계 요소다

이번 설정 파일에는 이런 주석이 있습니다.

① 판정 가능한 기준

“기사 하나를 보고 예/아니오로 답할 수 있는 질문이어야 한다.”

좋은 기준은 AI가 실제 기사 하나를 보고 **Yes/No로 판단할 수 있어야** 합니다.

“좋은 기사인가?”는 너무 모호합니다.

하지만

“지금 사용하는 API의 가격이 실제로 변경되었는가?”

는 비교적 명확합니다.

② 추상어보다 구체적인 표현

“홍보성”처럼 추상적인 표현보다 구체적인 사례를 적습니다.

이것은 AI에게도 중요하고 사람에게도 중요합니다.

③ 기준의 우선순위

“위에 있을수록 우선”

기준끼리 충돌할 경우 무엇을 우선할지 미리 정해놓는 것입니다.

따라서 좋은 설정 파일은 단순한 데이터 저장소가 아니라,

“이 값을 어떻게 작성하고 판단해야 하는가”까지 설명하는 문서가 됩니다.

6. 설정과 코드를 구분하는 방법

두 가지 질문을 던지면 됩니다.

질문 1

분야가 바뀌면 이 값도 바뀌는가?

YES라면 → 설정 후보

예:

독자
중요도 기준
토픽
톤
버릴 기사

질문 2

이 값을 수정할 사람이 개발자인가?

NO라면 → 코드에서 분리하는 것이 좋습니다.

예를 들어:

예선에서 40개를 묶어 처리한다

이것은 프로그램의 **절차**입니다.

반면

어떤 뉴스를 중요한 뉴스로 볼 것인가

는 **업무 기준**입니다.

따라서 둘을 분리합니다.

```
audience.yaml
↓
독자·기준·토픽
↓
Python 코드
↓
수집 → 선별 → 취재 → 검수 → 발행
```

7. 그렇다고 모든 것을 설정 파일로 빼면 안 된다

여기에도 중요한 균형이 있습니다.

모든 것을 YAML로 빼면:

```
설정 100개
+
코드 100개
```

가 되어 오히려 시스템을 이해하기 어려워집니다.

또 설정 조합이 너무 많아지면 테스트하기도 어려워집니다.

따라서 기준은 단순히

“바뀔 수 있는가?”

가 아니라

“실제로 자주 바뀌는가?”

에 가깝습니다.

자주 바뀌는 분야별 기준은 설정으로 빼고, 거의 변하지 않는 핵심 절차는 코드에 남기는 것입니다.

8. YAML의 약점은 Pydantic으로 보완한다

설정 파일의 약점은 오타나 잘못된 형식입니다.

예를 들어:

```
important_criteria
```

라고 해야 하는데

```
importent_criteria
```

라고 잘못 입력할 수 있습니다.

이런 문제를 **Pydantic** 같은 스키마 검증으로 잡습니다.

개념적으로는:

```
YAML 읽기
↓
Pydantic 검사
↓
문제 없음 → 프로그램 실행
문제 있음 → 즉시 오류
```

가 됩니다.

이것이 중요한 이유는 새벽 자동 실행 중간에 오류가 발생하는 것보다 프로그램 시작 전에 오류를 발견하는 것이 훨씬 좋기 때문입니다.

9. 이번 챕터에서 만들어진 구조

최종적으로는 이런 구조가 됩니다.

```
audience.yaml
|
├─ 독자
├─ 중요도 기준
├─ 버릴 것
└─ 토픽
    ↓
  Python 코드
```

↓
수집 → 예선 → 취재 → 검수 → 발행

그리고 분야가 바뀌면:

AI 뉴스
↓
audience.yaml만 변경
↓
채용 뉴스

처럼 바꿀 수 있습니다.

프로그램의 뼈대는 그대로이고 콘텐츠의 기준만 교체되는 것입니다.

10. 이 개념이 AI 개발에서 중요한 이유

이것은 단순한 YAML 사용법이 아닙니다.

핵심은 **재사용 가능한 AI 시스템**을 만드는 설계 원칙입니다.

잘못 만든 시스템:

AI 뉴스용 코드
AI 채용뉴스용 코드
AI 논문뉴스용 코드
AI 콘텐츠뉴스용 코드

각각 따로 만듭니다.

좋은 시스템:

공통 엔진 —┬─ AI 뉴스 설정
 ├─ 채용 뉴스 설정
 ├─ 논문 뉴스 설정
 └─ 콘텐츠 뉴스 설정

이렇게 됩니다.

즉,

하나의 엔진을 여러 분야에 재사용할 수 있습니다.

이것은 앞으로 사용자님께서 AI 컨설팅이나 MSM 같은 서비스를 만들 때도 매우 중요한 사고방식입니다.

객관식 정답

정답: ②

'홍보성'은 판정 기준이 사람마다 다른 추상어이지만, MOU·업무협약·투자유치·수상처럼 구체적인 유형을 적으면 해석의 여지가 줄어들기 때문입니다.

★ 꼭 기억해야 할 7가지

1. 구조와 내용을 분리한다.
2. 구조는 여러 분야에서 재사용할 수 있어야 한다.
3. 분야마다 달라지는 독자·기준·토픽은 **설정 파일**로 분리할 수 있다.
4. 누가 수정할 것인지가 설정 위치를 결정하는 중요한 기준이다.
5. 설정에는 **값뿐 아니라 작성 방법과 판단 규칙도 주석으로 남긴다.**
6. 추상적인 표현보다 **판정 가능한 구체적인 표현**이 좋다.
7. YAML의 오류 가능성은 **Pydantic 스키마 검증**으로 보완한다.

이번 챕터에서 한 단계 더 중요한 깨달음

앞선 챕터에서는 뉴스레터를 잘 작동하는 자동화 시스템으로 만들었다면,

이번 챕터부터는 그것을 다른 업무에도 재사용할 수 있는 시스템으로 발전시키고 있습니다.

```
Chapter 01~12
작동하는 뉴스레터 만들기
    ↓
Chapter 13
구조와 내용을 분리하기
    ↓
다른 분야에도 재사용
    ↓
AI 업무 자동화 엔진
```

이 관점은 앞으로 AI 컨설팅을 하실 때 특히 중요합니다.

고객마다 프로그램을 새로 만드는 것이 아니라,

공통 AI 업무 엔진 + 고객별 설정

으로 접근할 수 있기 때문입니다.

한 문장 요약

좋은 AI 시스템은 특정 분야에 맞춘 프로그램이 아니라, 공통적인 구조는 코드로 유지하고 분야별 기준과 내용은 설정으로 바꿔 쓸 수 있는 시스템입니다.

Chapter14

/강의정리 Chapter 14 — 만든 것

이번 챕터는 새로운 기능을 배우는 챕터라기보다 지금까지 만든 뉴스레터 시스템을 전체적으로 돌아보고, 무엇을 왜 그렇게 설계했는지 정리하는 회고 챕터입니다.

핵심 질문은 이것입니다.

“우리가 만든 구조가 다른 분야에서도 그대로 쓸 수 있는가?”

결론부터 말하면 그렇습니다.

AI 뉴스라는 내용은 바뀔 수 있지만, 그 안에서 사용한 설계 원칙은 다른 자동화 업무에도 적용할 수 있습니다.

1. 처음에는 빈 노드였지만 이제 하나의 시스템이 되었다

처음에는 그래프에 빈 노드 5개만 있었습니다.

- ① 수집
- ② 선별
- ③ 취재/작성
- ④ 검수
- ⑤ 발행

각 단계에서 하나씩 기능을 채워 넣었습니다.

그리고 매 단계가 끝날 때마다 전체 그래프를 실행했기 때문에,

“지금 어디까지 만들었고 어디에서 문제가 생겼는가?”

를 계속 확인할 수 있었습니다.

결과적으로 지금은

```
RSS
↓
수집
↓
예선
↓
본문 취재
↓
검수
↓
발행
↓
Discord
```

가 자동으로 이어지고, GitHub Actions를 이용해 **매일 자동 실행**까지 할 수 있는 상태가 되었습니다.

2. 이번 과정에서 배운 가장 중요한 것은 “선택의 과정”이다

각 단계에서는 처음부터 하나의 답을 정하지 않았습니다.

먼저 여러 선택지를 펼쳐 놓고,

무엇을 선택했는가보다 무엇을 버렸는가

를 확인했습니다.

예를 들어:

- 수집 경로
- 선별 구조
- 제약을 어디에 둘 것인가
- 출력 구조
- 검수 방식
- 뉴스가 부족할 때 처리 방법

- 발행 방법
- 설정을 어디에 둘 것인가

등을 하나씩 결정했습니다.

이런 방식이 중요한 이유는 나중에 문제가 생겼을 때

“왜 이렇게 만들었지?”

에 답할 수 있기 때문입니다.

3. ① 수집 — 관문과 등급은 다르다

뉴스 소스를 평가할 때 두 가지를 분리했습니다.

관문(Gate)

최소 조건을 만족하는가?

예를 들어 접근 가능한가, RSS가 정상인가 등입니다.

등급(Score)

여러 소스 중 어느 것이 더 좋은가?

입니다.

따라서

관문 통과
≠
좋은 소스

입니다.

관문은 탈락시키기 위한 기준, 등급은 우열을 가리기 위한 기준입니다.

이 둘을 합쳐버리면 서로 다른 성격의 판단이 섞이게 됩니다.

4. ② 선별 — 고정된 개수는 “채점”이 아니라 “정렬” 문제

예를 들어 후보 40개 중 최종 5개를 뽑아야 한다면,
각 후보를 독립적으로

“좋다 / 나쁘다”

라고 채점하는 것만으로는 부족합니다.

결국 필요한 것은

“40개 중 어떤 5개가 더 중요한가?”

라는 **상대적인 비교**입니다.

즉,

후보 40개
↓
서로 비교
↓
순위
↓
상위 5개

라는 구조가 필요합니다.

또 후보가 너무 많으면 묶음으로 나누게 되는데, 이 경우 **묶음 사이의 비교를 일부 포기하는 비용**이 생깁니다.

그리고 여기서 중요한 교훈도 하나 얻었습니다.

프롬프트에 부탁한다고 해서 AI가 항상 그 지시를 정확하게 지키는 것은 아니다.

그래서 중요한 조건은 코드와 검증으로 보완해야 합니다.

5. ③ 요약 — 출력 구조가 검증 가능성을 결정한다

취재 결과를 하나의 긴 문장으로 만들지 않고

headline
summary
why

처럼 나뉘었습니다.

처음에는 단순히 보기 좋은 형식처럼 보이지만 실제로는 훨씬 중요한 의미가 있습니다.

각 항목을 분리했기 때문에

```
headline → 원문과 비교
summary  → 원문과 비교
why      → 해석 영역
```

처럼 각각 다른 방식으로 검사할 수 있습니다.

즉,

출력 형식을 설계하는 것은 동시에 검증 방법을 설계하는 일입니다.

6. ④ 검수 — 핵심은 “무엇과 무엇을 비교하는가”

검수 단계에서 숫자나 문자열만 단순 비교하는 방법을 시험했습니다.

하지만

```
three months
↓
3개월
```

처럼 번역이나 단위 변환이 들어가면 정상적인 내용도 오류로 판단할 수 있었습니다.

따라서 문제는

검사를 했느냐 안 했느냐

가 아니라

문제에 맞는 대조 방법을 선택했느냐

였습니다.

결국 원문과 생성 결과를 함께 LLM에게 보여주고 검증하는 방식을 선택했습니다.

7. 뼈대 — 모든 단계를 하나의 노드로 만들 필요는 없다

모든 단계가 똑같이 독립적인 노드가 될 필요는 없습니다.

중요한 기준은:

“이 단계가 다른 항목을 봐야 답할 수 있는가?”

입니다.

다른 후보를 비교해야 한다면 독립적인 단계가 필요할 수 있고, 단순한 처리라면 합쳐도 됩니다.

그리고 뉴스가 5건보다 적다고 해서 억지로 다시 수집하지 않았습니다.

3건만 있으면
→ 3건 발행

으로 처리했습니다.

이런 단순한 규칙 하나가 복잡한 재시도 설계 전체를 없애는 효과를 냅니다.

8. ⑤ 발행 — 외부에 영향을 주는 단계는 특별하게 다룬다

발행은 다른 단계와 다릅니다.

수집이나 요약은 실패해도 내부에서 끝나지만,

발행은 실제 사람에게 전달됩니다.

따라서 `dry_run` 을 먼저 만들었습니다.

실제 발행
↑
`dry_run`으로 먼저 확인

그리고 발행 단계에는 불필요하게 LLM을 넣지 않았습니다.

마지막으로 실제 Discord에서 받아보는 것까지 확인했습니다.

즉,

“코드가 실행된다”가 아니라 “실제 결과가 외부에 전달된다”까지 만든 것입니다.

9. 측정 — 만든 시스템을 다시 평가할 수 있게 했다

Chapter 12에서 만든 `metrics.jsonl` 도 중요한 결과물입니다.

실행마다

```
collected
picked
drafted
published
by_source
log
```

등을 기록합니다.

이를 통해

```
수집 → 선별 → 취재 → 발행
```

의 깔때기를 볼 수 있습니다.

즉, 이번 과정에서는 단순히 자동화하는 데서 끝나지 않고

자동화 → 측정 → 검증 → 개선

이라는 구조까지 만들었습니다.

10. 아직 해결하지 않은 문제도 있다

이번 프로젝트가 완성된 제품이라는 뜻은 아닙니다.

① 평가셋과 회귀 테스트

현재는

“이번에 더 좋아진 것 같다.”

정도로 판단했습니다.

더 제대로 하려면 30~50개 정도의 기사에 사람이 정답을 붙여 **평가셋**을 만들고, 프롬프트를 변경할 때마다 점수를 비교해야 합니다.

그래야

| A는 좋아졌는데 B는 나빠진 문제

를 발견할 수 있습니다.

② 사람의 승인

현재는

AI
↓
검수
↓
자동 발행

입니다.

실무에서는

AI
↓
검수
↓
사람 승인
↓
발행

구조가 필요할 수도 있습니다.

이 경우 그래프를 **멈췄다가 다시 시작하는** 구조가 필요합니다.

③ 비용과 속도 최적화

현재 모든 단계에서 비슷한 모델을 사용합니다.

하지만 예선처럼 단순한 판단에는 저렴하고 빠른 모델을 사용하고, 복잡한 취재나 검수에는 더 좋은 모델을 사용하는 방식으로 비용을 줄일 수 있습니다.

단순 작업 → 저렴한 모델
복잡한 작업 → 고성능 모델

이것이 실제 AI 서비스 운영에서 중요한 **모델 라우팅/비용 최적화**입니다.

④ 소스 확장과 스크래핑

현재는 RSS 중심입니다.

RSS가 없는 분야를 다루려면 웹페이지 수집이나 스크래핑이 필요하고, 이때는 `robots.txt`, 페이지 구조 변화, 접근 제한 등의 문제까지 고려해야 합니다.

11. 이번 챕터의 가장 큰 의미

처음에는

“AI 뉴스를 하나 만들어 보자.”

에서 시작했습니다.

하지만 끝까지 따라오면 실제로 배운 것은 뉴스레터 만드는 법보다 훨씬 넓습니다.

문제 정의
↓
선택지 비교
↓
구조 설계
↓
AI 노드 구현
↓
검증
↓
외부 발행
↓
자동 실행
↓
측정
↓
회고 및 개선

이것이 바로 **AI 에이전트/자동화 시스템**을 설계하는 사고방식입니다.

주관식 퀴즈에 대한 생각

사용자님이 지금까지 공부해 오신 방향을 기준으로 생각하면, 이 질문은 다음처럼 접근하시면 좋습니다.

예를 들어 앞으로 **콘텐츠 산업 뉴스레터**로 바꾼다고 생각해 보겠습니다.

가장 크게 손볼 단계

선별 단계가 될 가능성이 큼니다.

AI 뉴스에서는

| 모델·API 가격 변화, 개발 방식 변화

가 중요하지만 콘텐츠 산업에서는

| 새로운 콘텐츠 IP, 플랫폼 변화, 제작 방식 변화, 해외 진출, 유통 구조 변화

등이 중요하기 때문입니다.

즉 선별 기준 자체가 크게 달라집니다.

그대로 사용할 수 있는 단계

반면

수집

→ 후보 선별

→ 본문 취재

→ 검수

→ 발행

이라는 전체 구조 자체는 거의 그대로 재사용할 수 있습니다.

이것이 바로 Chapter 13에서 배운 “구조와 내용의 분리”가 실제로 의미하는 것입니다.

★ 꼭 기억할 핵심 6가지

1. 빈 노드부터 시작해서 한 단계씩 완성한다.
2. 선택할 때는 무엇을 골랐는지뿐 아니라 무엇을 버렸는지 기록한다.
3. 관문·선별·검수·발행은 각각 다른 문제를 해결한다.
4. 출력 형식을 어떻게 설계하느냐가 나중의 검증 가능성을 결정한다.
5. 자동화는 실행으로 끝나는 것이 아니라 측정하고 다시 결정하는 과정까지 포함한다.
6. 좋은 시스템은 특정 업무에만 맞는 시스템이 아니라 구조를 재사용하고 내용만 교체할 수 있는 시스템이다.

 이번 과정 전체를 한 단계 높여서 보면

사용자님이 지금 배우고 계신 것은 단순한 “LangGraph 코딩”이 아닙니다.

더 중요한 것은 다음 사고방식입니다.

사람이 하던 업무를 → 단계로 분해하고 → AI에게 맡기고 → 규칙으로 통제하고 → 결과를 검수하고 → 자동 실행하고 → 데이터를 측정해서 다시 개선하는 것

이것이 앞으로 사용자님이 하시려는 AI 컨설팅과 1인 창업에서 가장 중요한 역량 중 하나입니다.

한 문장 요약

AI 자동화의 완성은 “돌아가는 프로그램”이 아니라, 업무 구조를 재사용할 수 있고 실행 결과를 측정하여 계속 개선할 수 있는 시스템을 만드는 것입니다.